Greedy Algorithms

Prof. Timothy Sun

Cashier's algorithm

"Take biggest possible coin until you're done."

```
CASHIER(C)
    while C > 0
        if C ≥ 25
            take quarter
        else if C ≥ 10
            take dime
        else if C ≥ 5
            take nickel
        else
            take penny
```

Cashier's algorithm

"Take biggest possible coin until you're done."

Claim: matches DP solution

```
CASHIER(C)
    while C > 0
        if C ≥ 25
            take quarter
        else if C ≥ 10
            take dime
        else if C ≥ 5
            take nickel
        else
            take penny
```

Cashier's algorithm

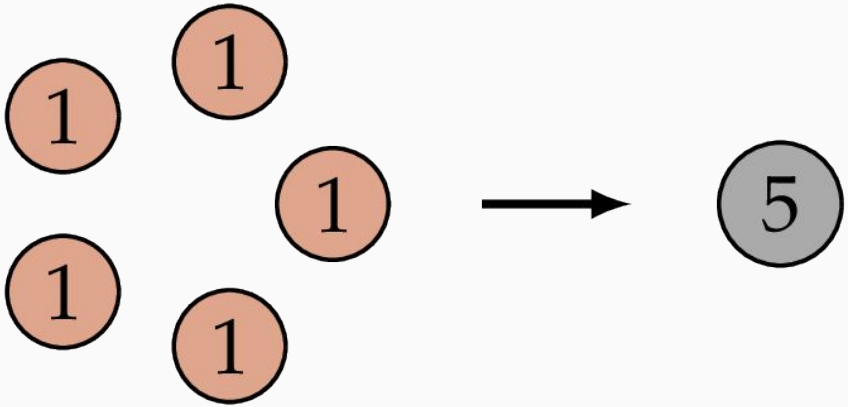
"Take biggest possible coin until you're done."

Claim: matches DP solution (for this currency)

```
CASHIER(C)
    while C > 0
        if C ≥ 25
            take quarter
        else if C ≥ 10
            take dime
        else if C ≥ 5
            take nickel
        else
            take penny
```

Cashier's algorithm

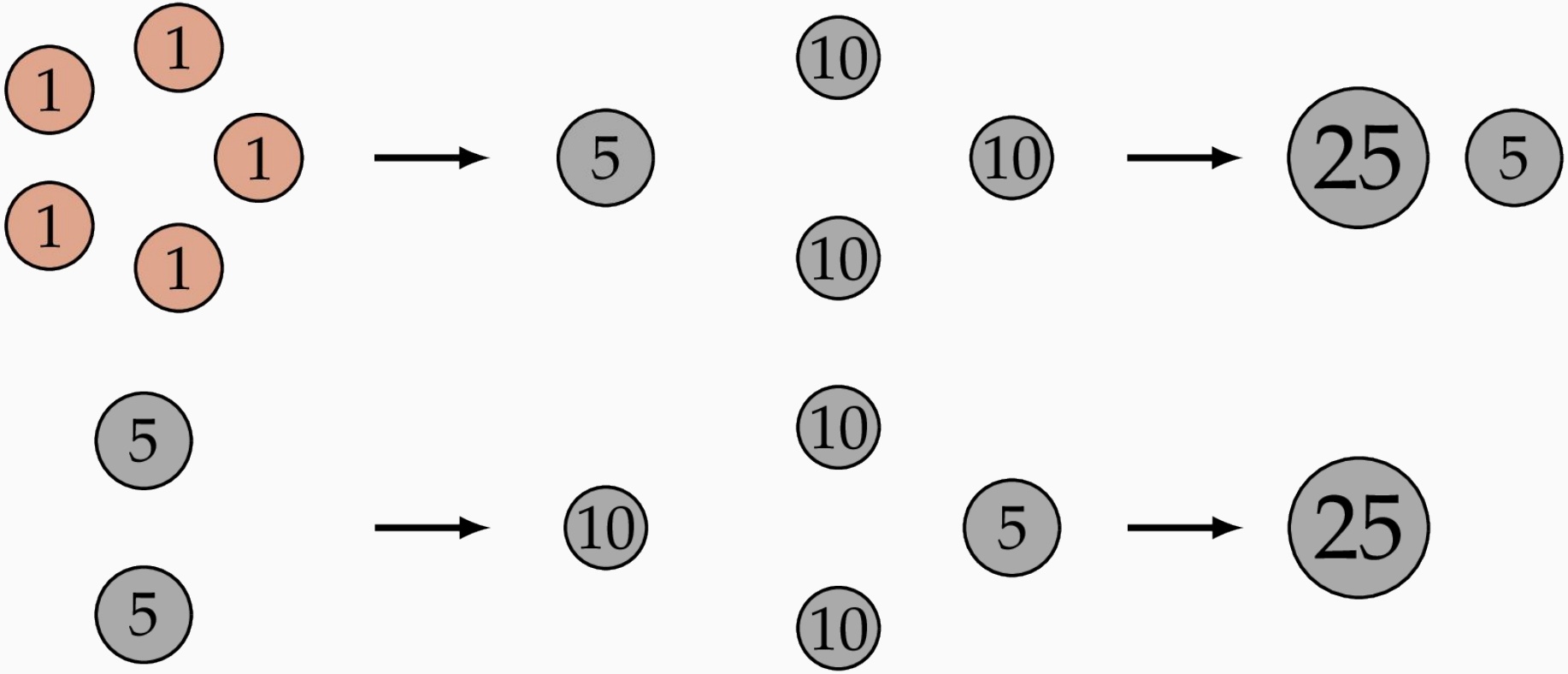
In an optimal solution, you can't have more than 4 pennies:




```
CASHIER(C)
    while C > 0
        if C ≥ 25
            take quarter
        else if C ≥ 10
            take dime
        else if C ≥ 5
            take nickel
        else
            take penny
```

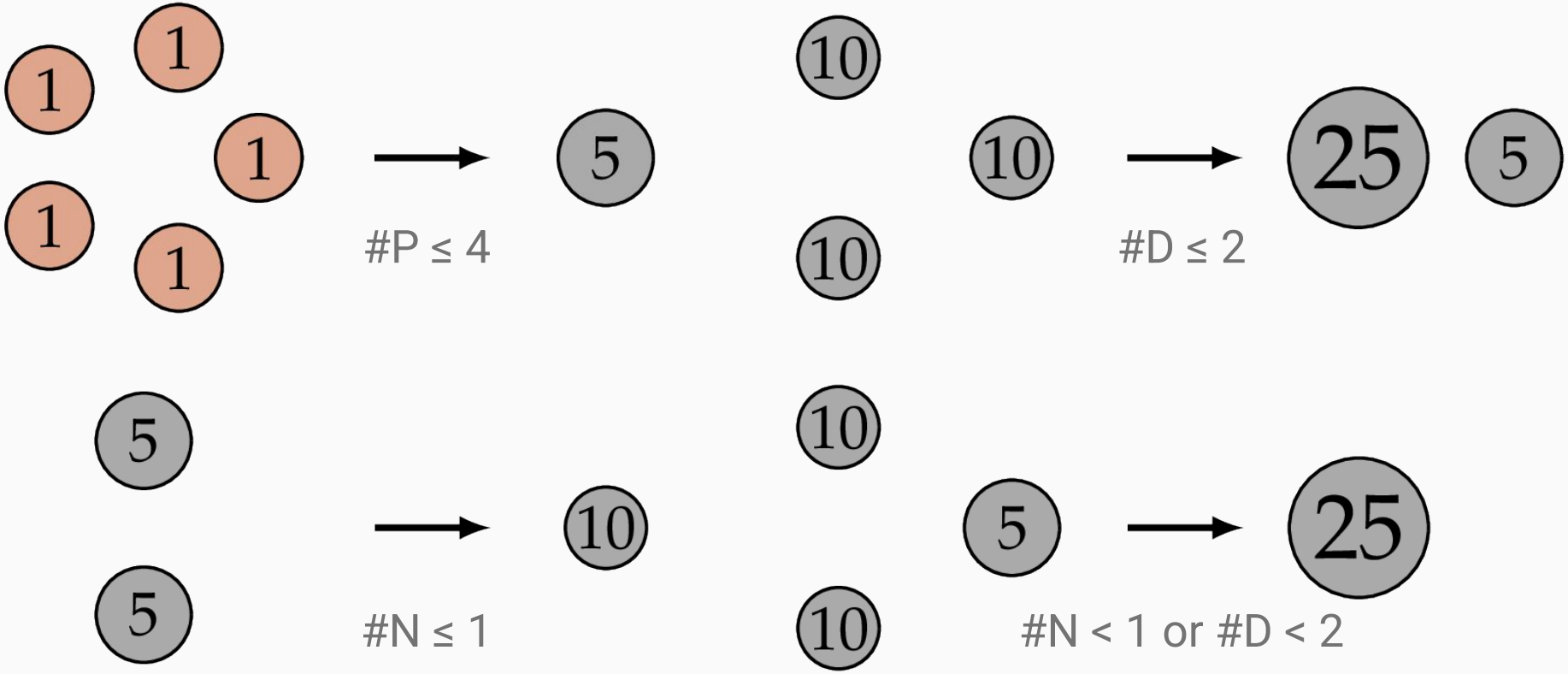
Cashier's algorithm

The greedy choice can be shown to always be correct with many ***exchanges***:



Cashier's algorithm

The greedy choice can be shown to always be correct with many ***exchanges***:



Cashier’s algorithm

Why is the cashier’s algorithm not optimal for 1¢, 5¢, 7¢?

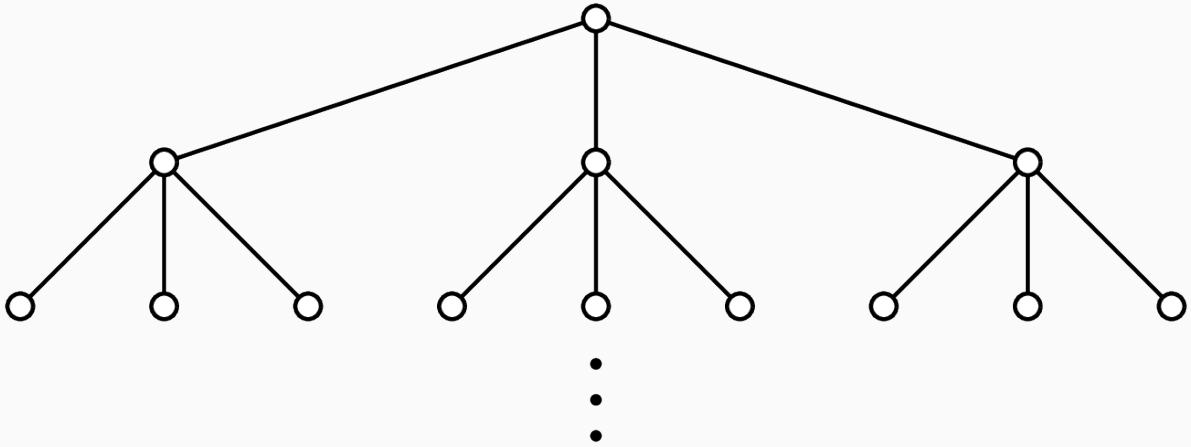
Cashier’s algorithm

Why is the cashier’s algorithm not optimal for 1¢, 5¢, 7¢?

10 = 5 + 5 (optimal) = 7 + 1 + 1 + 1 (greedy)

Backtracking vs. Greedy

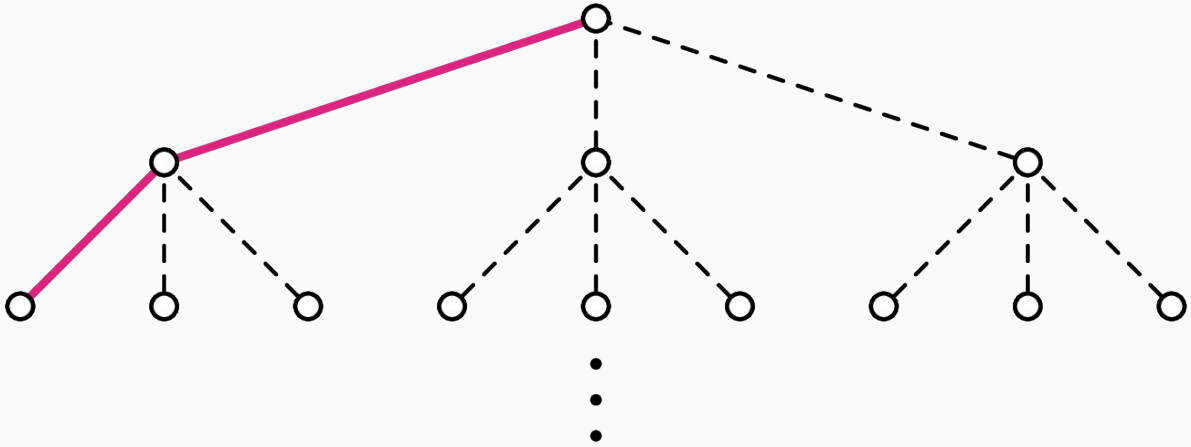
Backtracking: try every branch



Backtracking vs. Greedy

Backtracking: try every branch

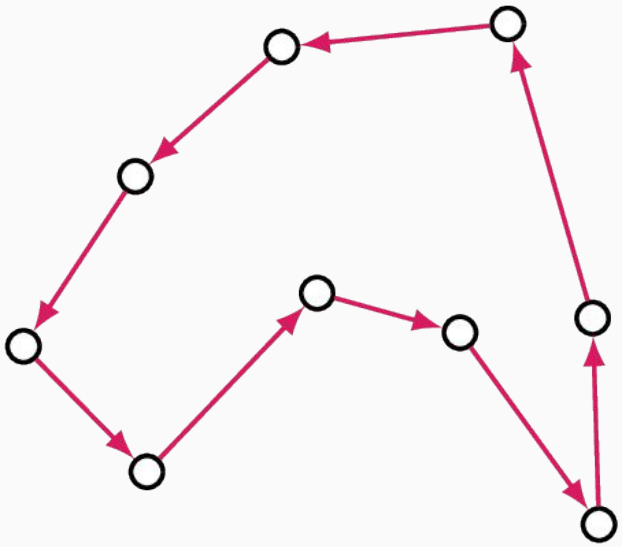
Greedy: try the “best-looking” branch (**often, just a heuristic**)



Greedy TSP

Traveling Salesperson Problem:

“What’s the quickest way to visit every city and come back to where you started?”

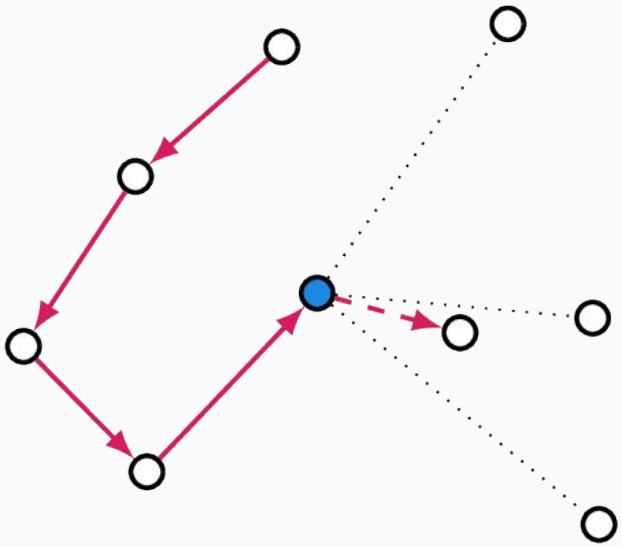


Greedy TSP

Nearest neighbor heuristic

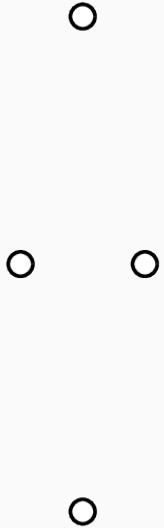
- “Go to closest unvisited city.”

Very rarely produces the optimal solution.

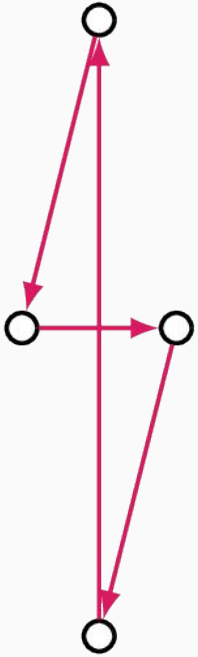


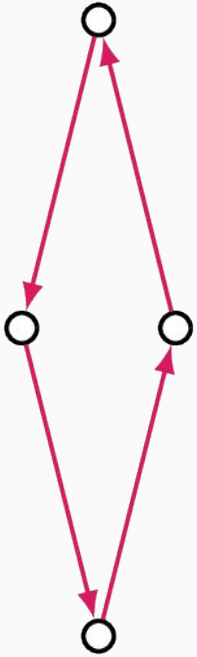
Greedy TSP

In the following configuration, how does nearest-neighbor perform?



Greedy TSP

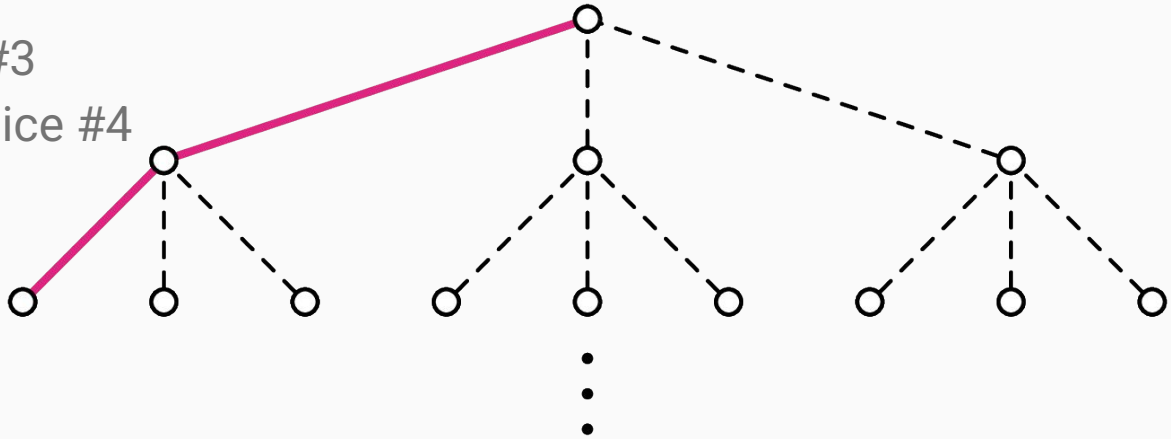




Proving correctness

Backtracking considers all branches, but greedy algorithms only consider one:

- Greedy choice #1
  - Greedy choice #2
    - Greedy choice #3
      - Greedy choice #4
        - ...



Proving correctness

1. “Induction”
    a. There is an optimal solution containing greedy choice #1
    b. Recurse and use induction
2. “Well-ordering”
    a. Look at first difference between greedy and optimal solutions
    b. The optimal solution can actually use the greedy choice here

Proving correctness

1. “Induction”
    a. There is an optimal solution containing greedy choice #1
    b. Recurse and use induction
2. “Well-ordering”
    a. Look at first difference between greedy and optimal solutions
    b. The optimal solution can actually use the greedy choice here

Both approaches use an **exchange** argument.

“Unweighted” 0-1 Knapsack

Suppose you have a bag with capacity ***C*** and a set of items with weights

$$w_1 < w_2 < w_3 < \ldots < w_n$$

Maximize the number of items that can be placed into the bag without exceeding ***C***.

“Unweighted” 0-1 Knapsack

Greedy algorithm: pick $\boldsymbol{w}_1, \boldsymbol{w}_2, \boldsymbol{w}_3$… until you run out of space.

“Unweighted” 0-1 Knapsack

Greedy algorithm: pick $\boldsymbol{w}_1, \boldsymbol{w}_2, \boldsymbol{w}_3$… until you run out of space.

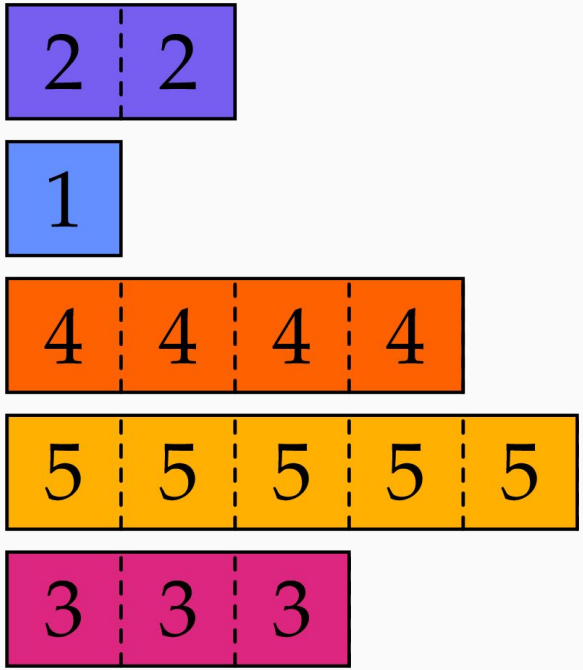
Proof: (“Induction” method) consider any optimal solution.

- If $\boldsymbol{w}_1$ is in it, we’re done.
- If not, ***exchange*** any element with $\boldsymbol{w}_1$.

File Storage

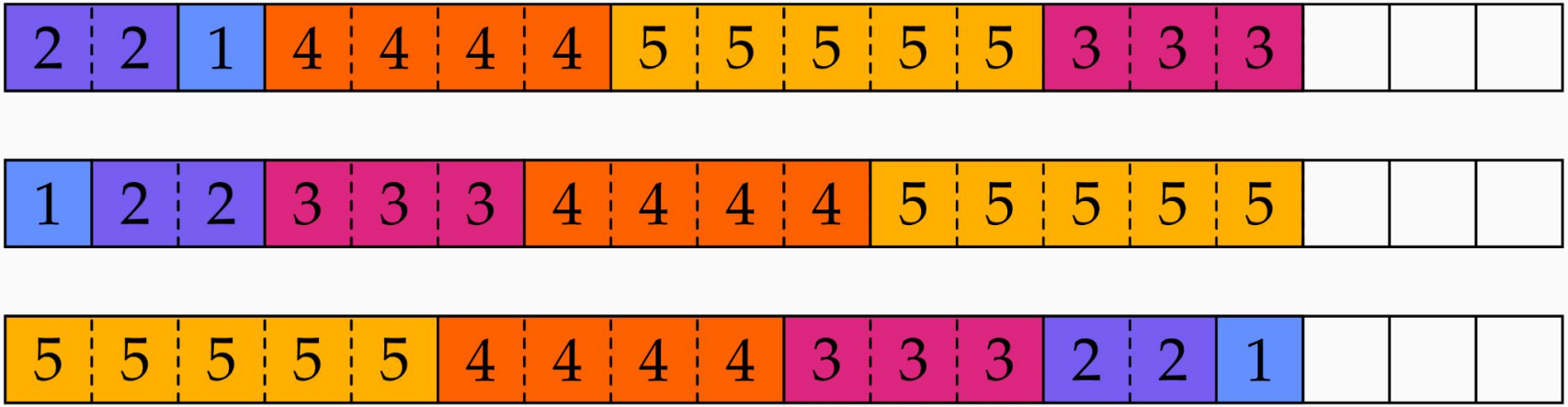
On a magnetic tape, data is found via ***linear search***.

What's the best way of storing, minimizing ***average*** search time (average starting index)?



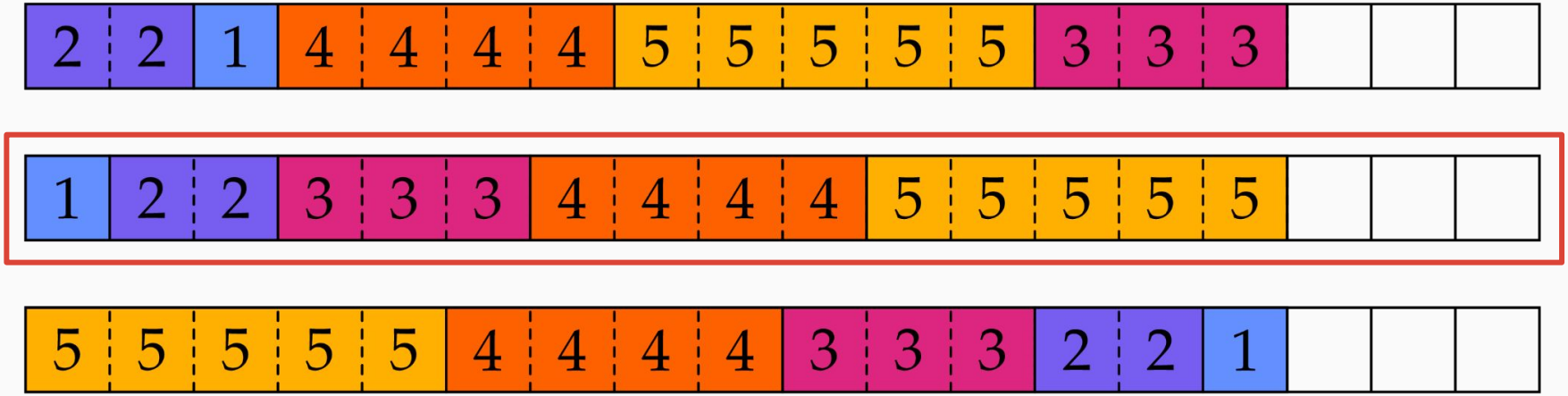
File Storage

Depending on the order, you can have different search times.



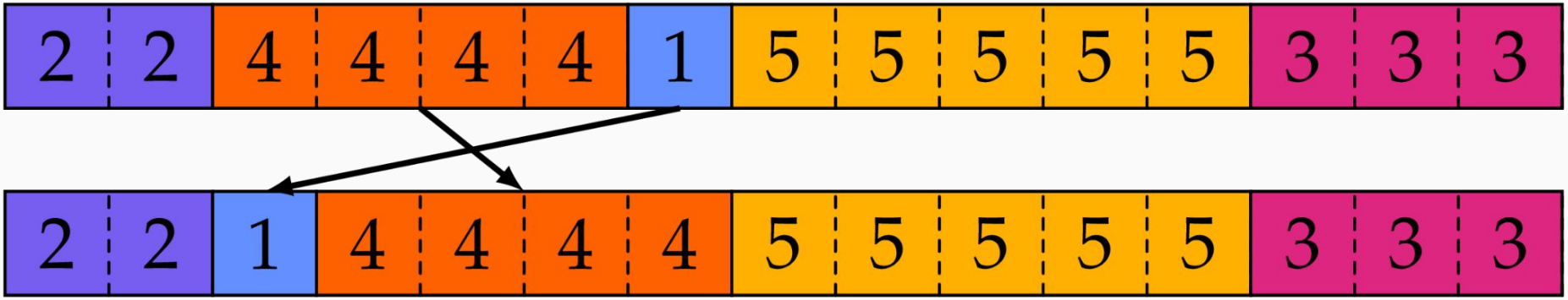
File Storage

The greedy algorithm (smallest → largest) wins!



File Storage

Why does the greedy approach work? “**Sorting**” reduces cost:



Ski rentals

When choosing skis, the length should be close to your height.

Ski rentals

When choosing skis, the length should be close to your height.

Given people's heights $P[1, \ldots, n]$ and ski lengths $S[1, \ldots, n]$, what's the best way of assigning skis so that the ***average*** difference between ski length and person height is minimized?

Ski rentals

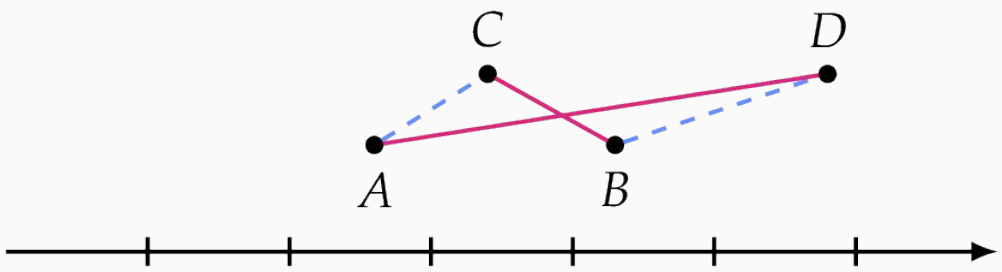
When choosing skis, the length should be close to your height.

Given people's heights $P[1, \ldots, n]$ and ski lengths $S[1, \ldots, n]$, what's the best way of assigning skis so that the ***average*** difference between ski length and person height is minimized?

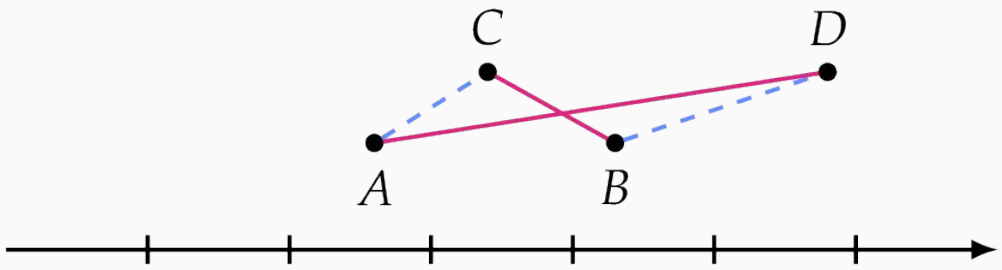
Greedy algorithm is again ***sorting***, but why does it work?

Ski rentals

The same “swap” argument holds: given skiers ***A*** and ***B***, and skis ***C*** and ***D***, the greedy way should be at least as good as the non-greedy way:



Ski rentals

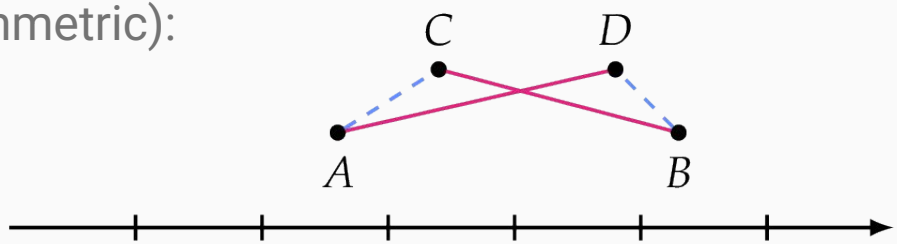


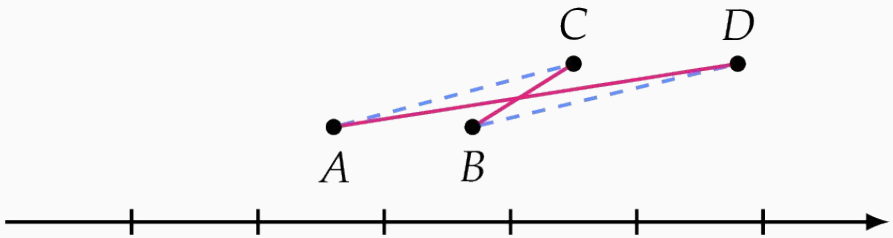
$$(D\text{-}A) + (B\text{-}C) = [(D\text{-}B)+(B\text{-}C)+(C\text{-}A)] + (B\text{-}C)$$

$$\geq (D\text{-}B) + (C\text{-}A)$$

Ski rentals

Two other cases to consider (others are symmetric):



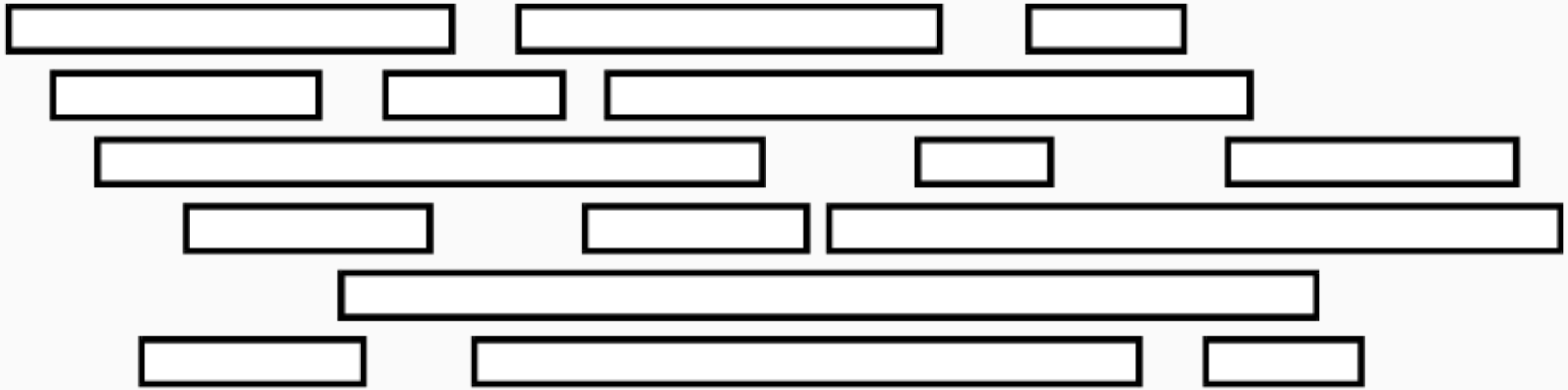


Class scheduling

Given a set of classes with start and finish times:

$$(s_1, f_1), (s_2, f_2), \ldots, (s_n, f_n)$$

Maximize the number of classes you can take without overlap.



Class scheduling

Greedy algorithm #1: pick shortest class

Greedy algorithm #2: pick longest class

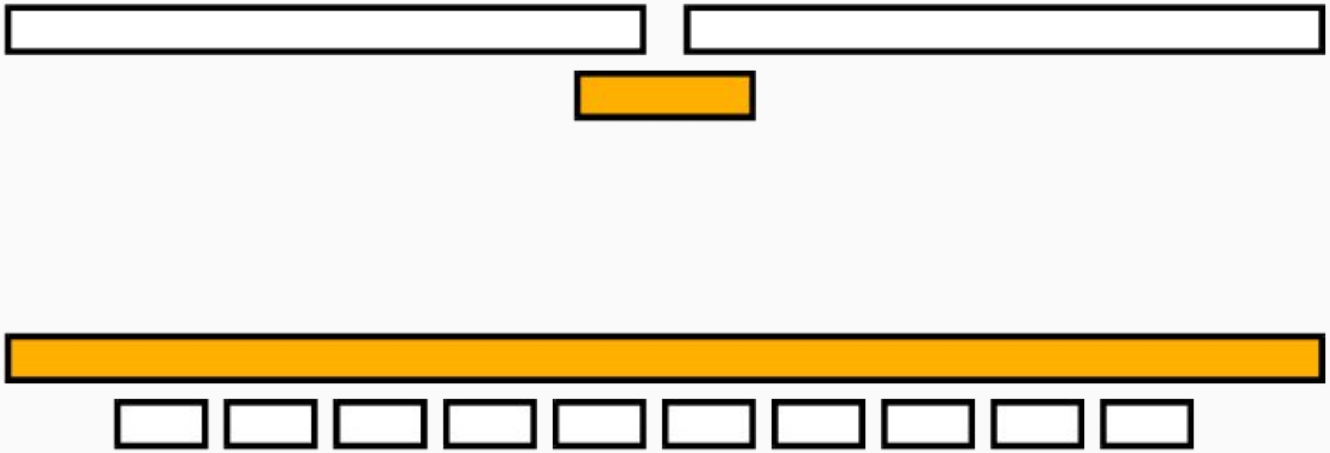
Greedy algorithm #3: pick earliest starting class

Greedy algorithm #4: pick earliest finishing class

...then remove any overlaps and recurse

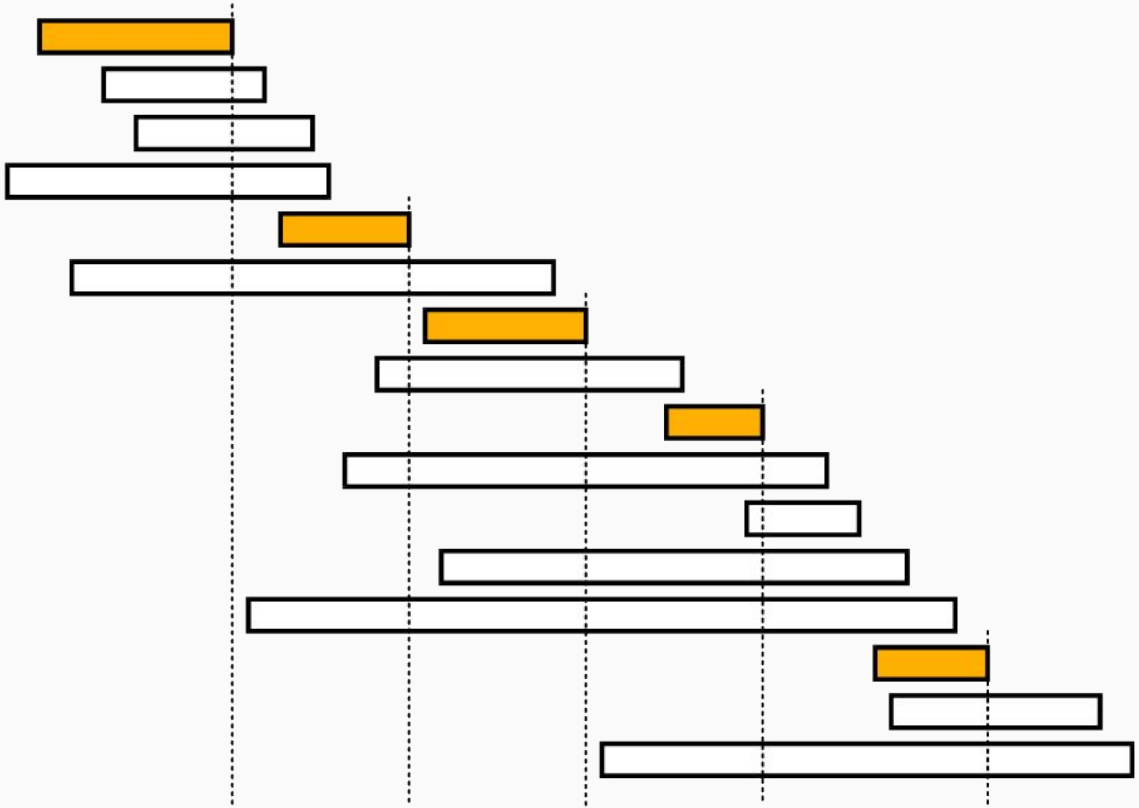
Class scheduling

Counterexamples for #1-#3:



The correct approach

Greedy algorithm #4: pick earliest finishing class

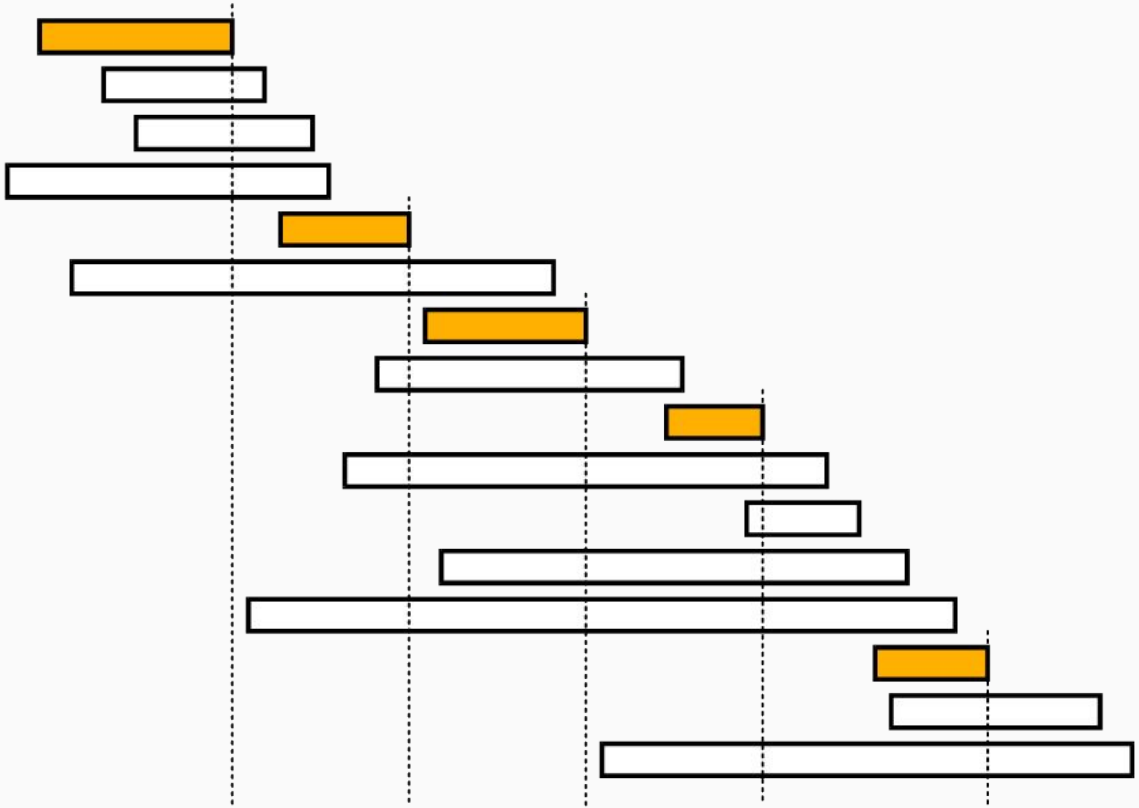


The correct approach

Greedy algorithm #4: pick earliest finishing class

Proof: another ***exchange*** argument, with key idea:

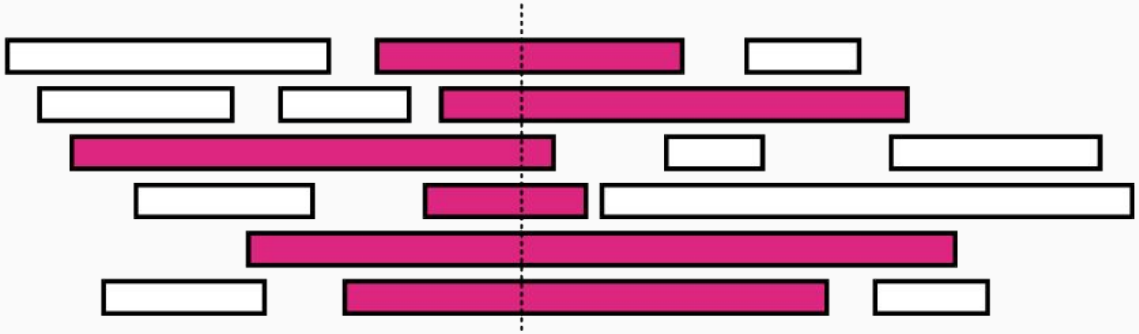
In a valid schedule, the first class ends before any other class begins.



Fire drills

The university wants to schedule fire drills so that every class can practice their evacuation plan.

What's the ***fewest*** number of drills that need to be scheduled?

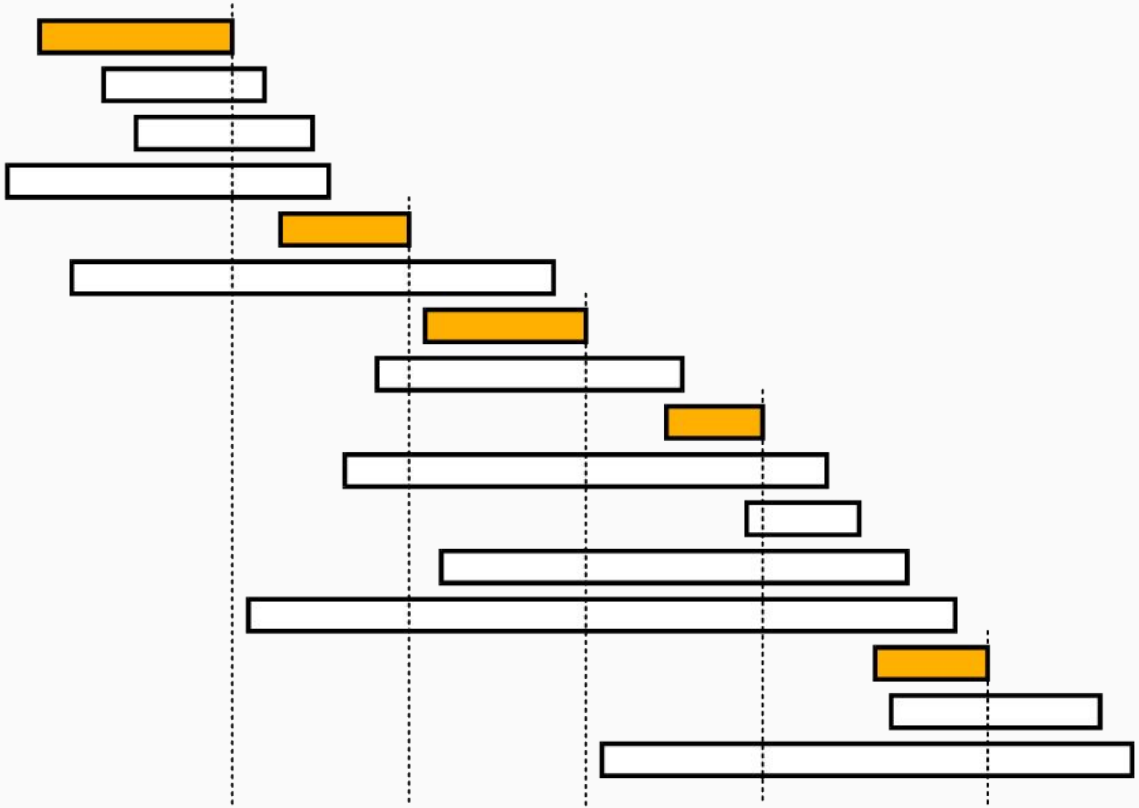


Fire drills

The university wants to schedule fire drills so that every class can practice their evacuation plan.

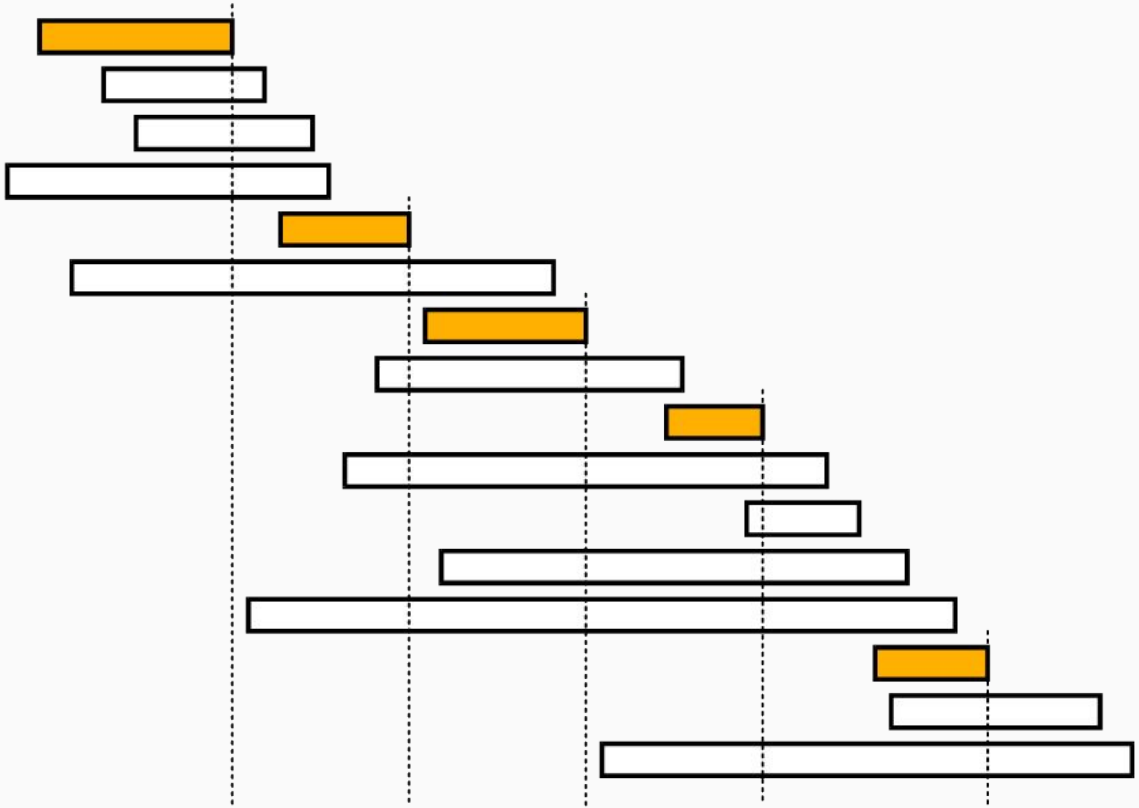
What's the ***fewest*** number of drills that need to be scheduled?

Use the same algorithm.



Proof by duality

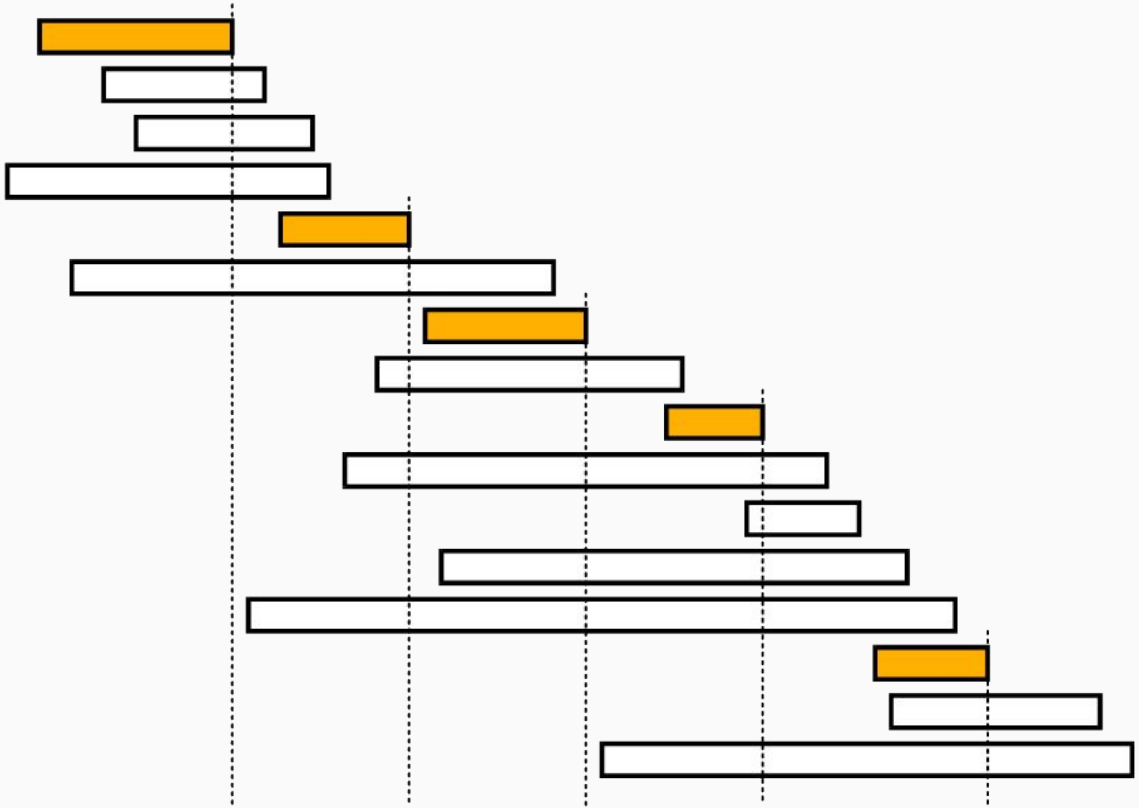
The number of fire drills can’t be less than the number of classes in a schedule.



Proof by duality

The number of fire drills can't be less than the number of classes in a schedule.

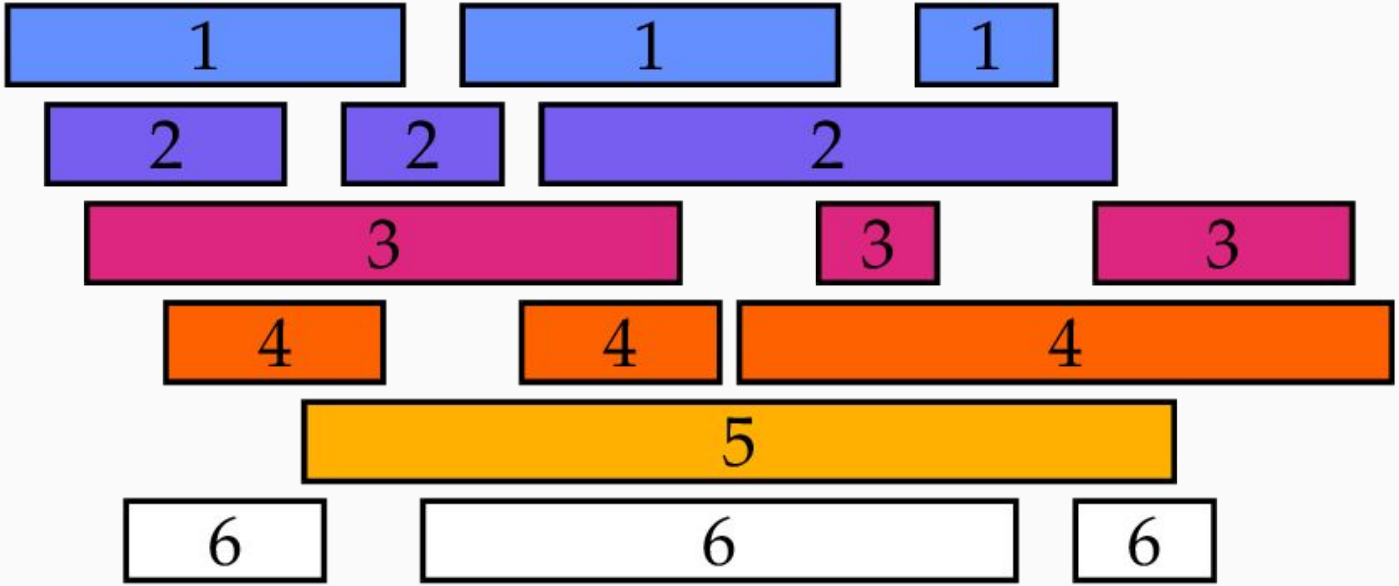
This plan uses the ***same number*** of drills as classes in our ***optimal*** schedule.



Assigning classrooms

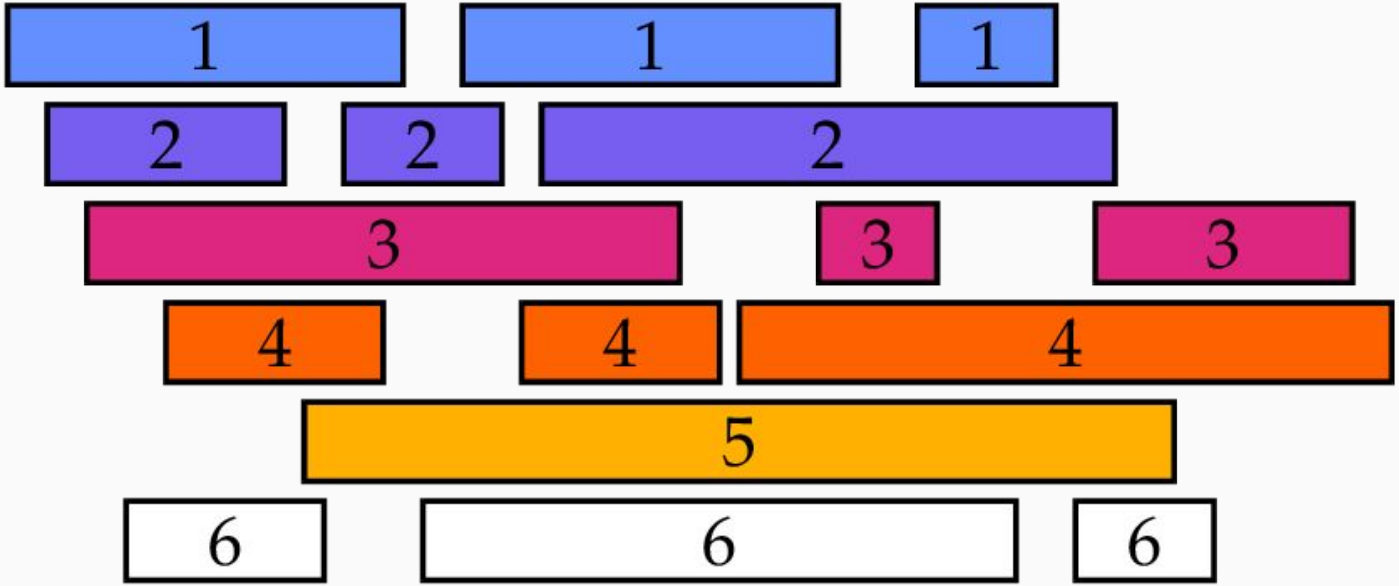
The university now wants to figure out which classes should be in which classrooms.

What's the ***fewest*** number of classrooms needed?



Assigning classrooms

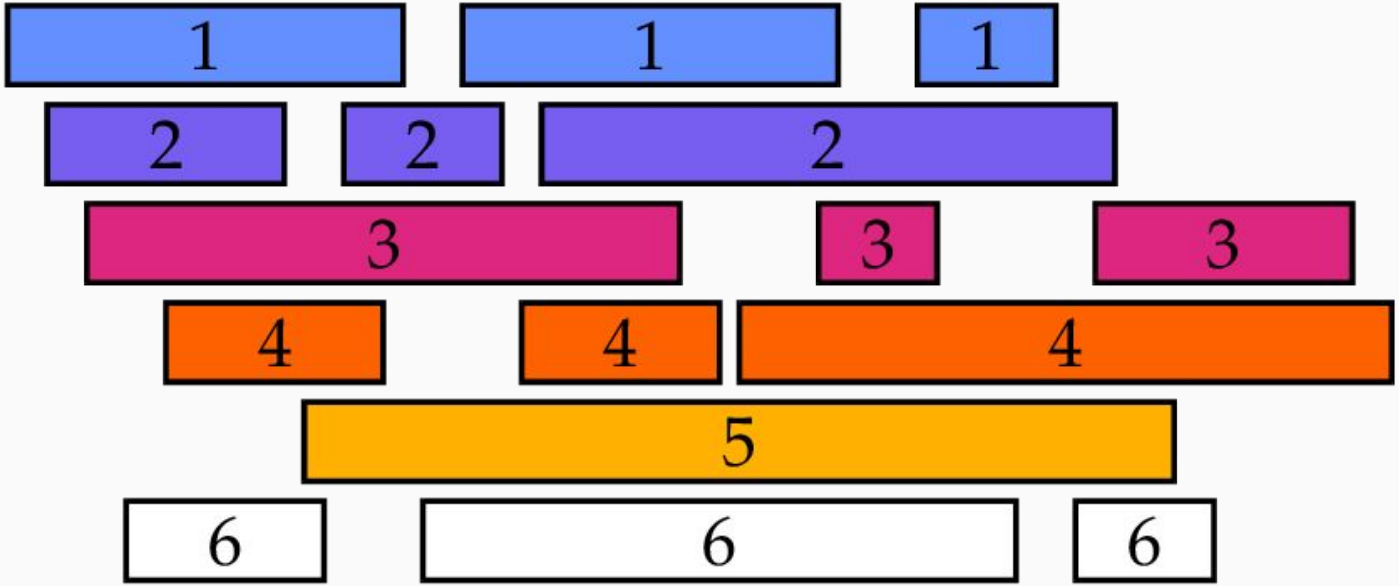
What would be a ***greedy*** way of assigning a color/room??



Assigning classrooms

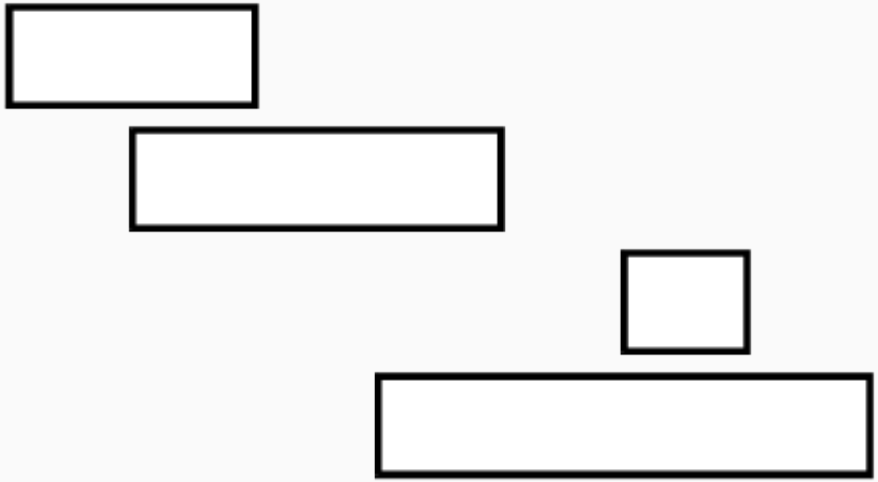
What would be a ***greedy*** way of assigning a color/room??

- When trying to color a class, give it the “smallest” color/room.



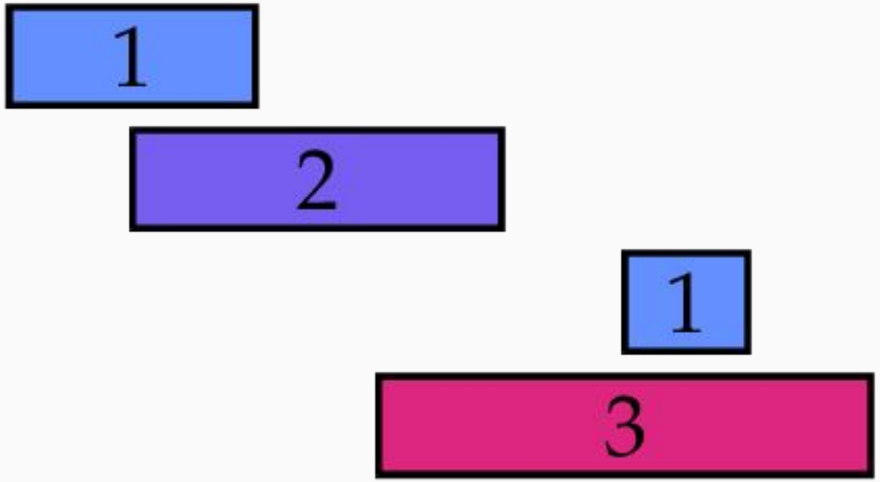
A suboptimal coloring strategy

The same approach from earlier doesn’t work:



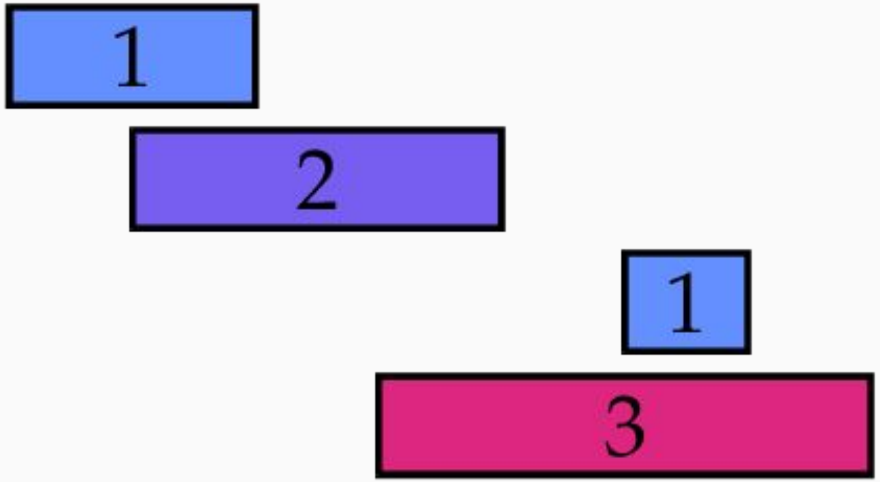
A suboptimal coloring strategy

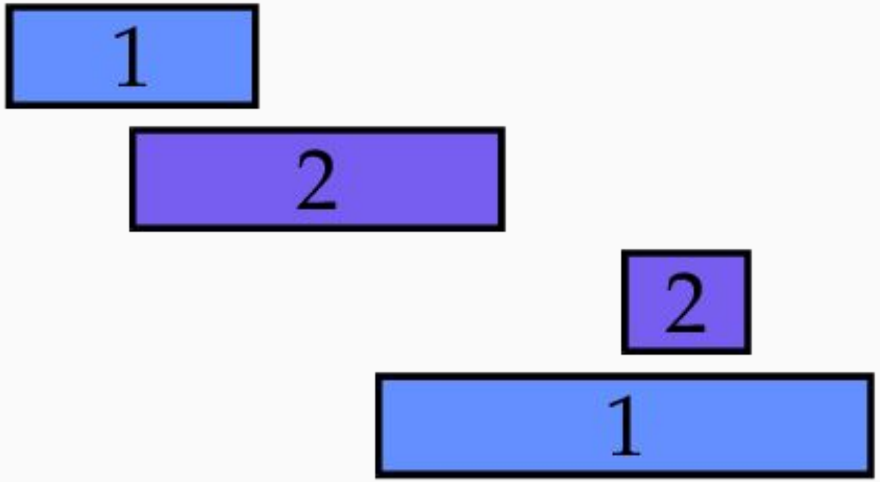
The same approach from earlier doesn’t work:



A suboptimal coloring strategy

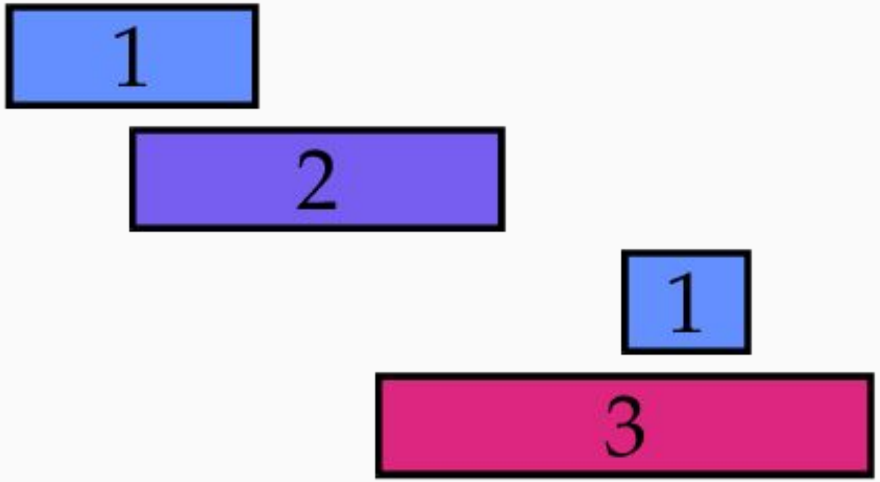
The same approach from earlier doesn't work:

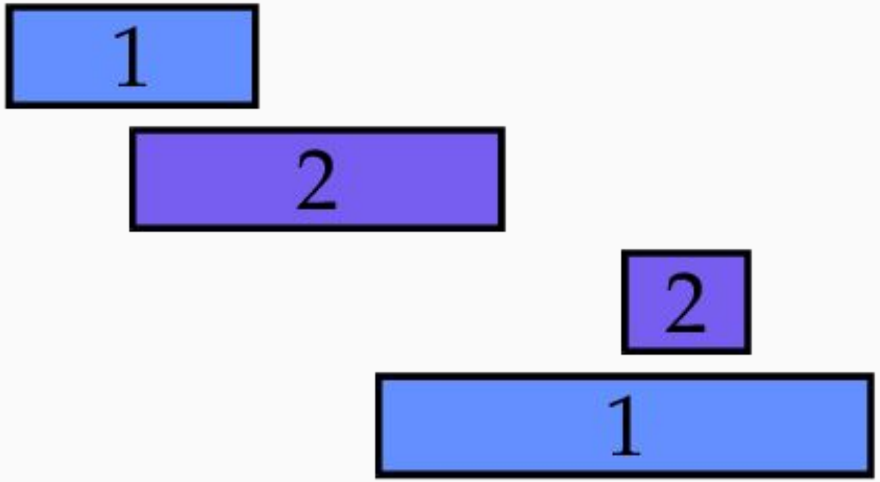




A suboptimal coloring strategy

The same approach from earlier doesn't work:

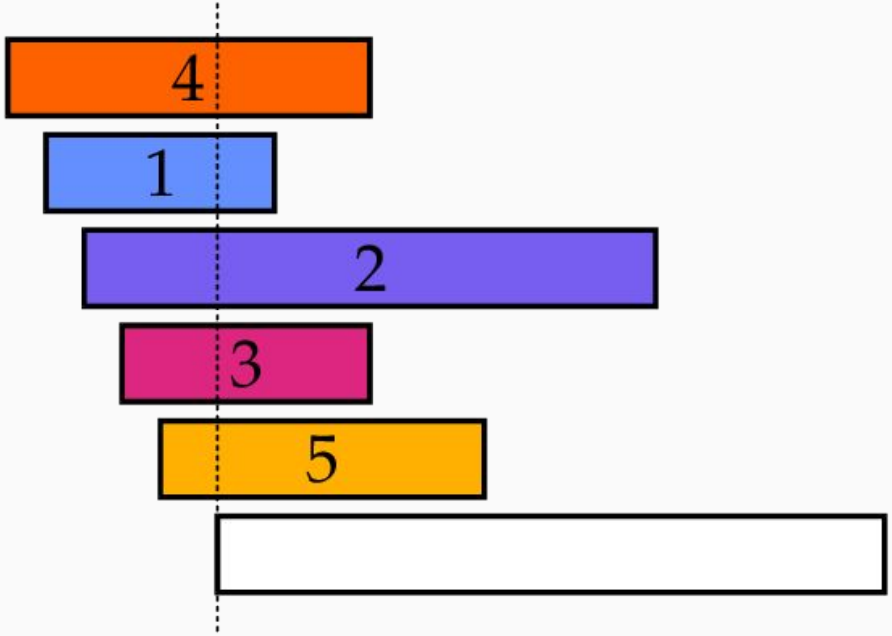




...but sorting by ***starting*** time does work!

Assigning classrooms

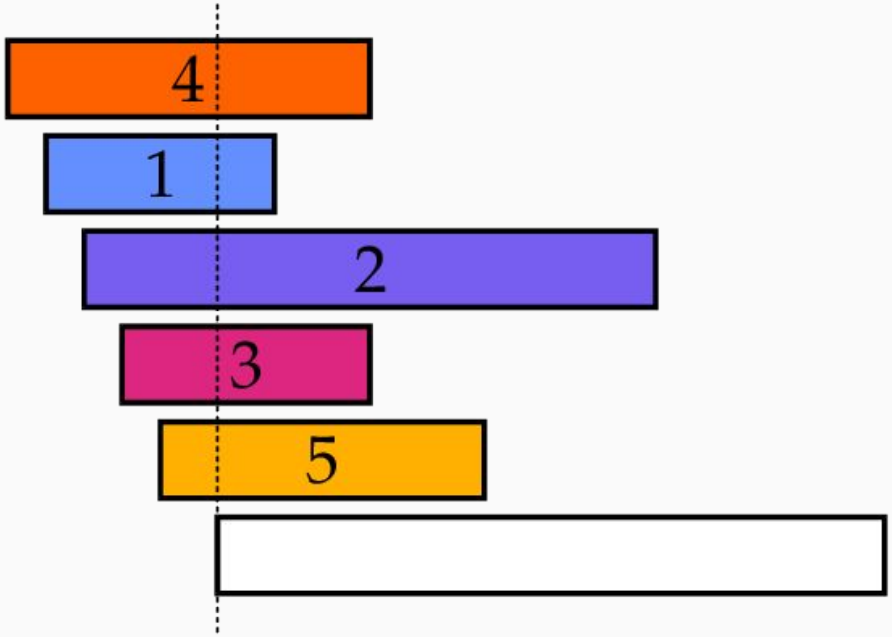
When going by increasing starting time, each time you need color ***k***, there are ***k*** classes going on at the same time.



Assigning classrooms

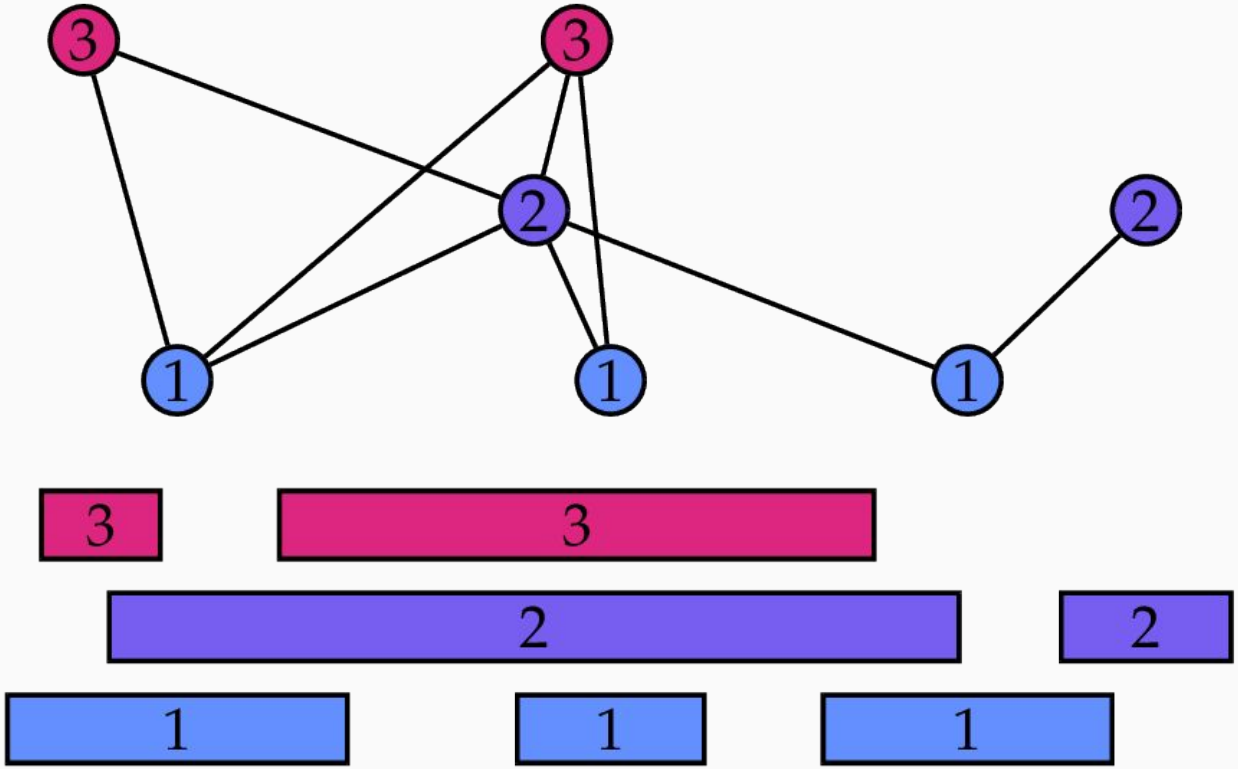
When going by increasing starting time, each time you need color ***k***, there are ***k*** classes going on at the same time.

Can't use fewer than ***k*** colors, so it must be optimal.



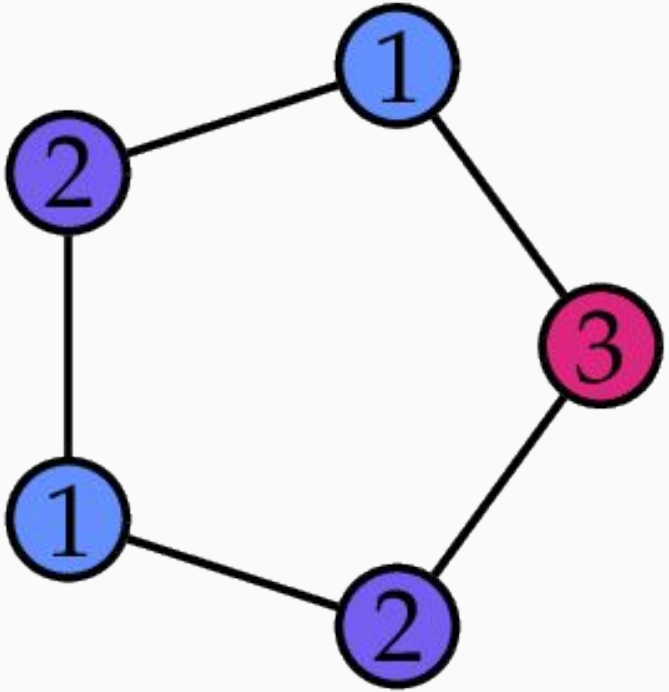
Interval graphs

This is a special instance of an NP-hard problem called ***graph coloring***.



Interval graphs

For general graphs, it’s harder to solve these problems.



Text compression

Given a long piece of text, want to compress the text:

- char primitive in Java: 2 bytes = 16 bits

Text compression

Given a long piece of text, want to compress the text:

- char primitive in Java: 2 bytes = 16 bits

1. Not every Unicode character used
2. Letter frequencies

Text compression

Suppose text only uses capital letters

- 26 characters → 5 bits
  - `A = 00000`
  - `B = 00001`
  - `C = 00010`
  - etc.

Text compression

To encode, replace each char with its codeword:

“BCAD” → 00001000100000000011

Text compression

To encode, replace each char with its codeword:

“BCAD” → 00001000100000000011

Doesn’t take into account letter frequencies

Decoding

- With a fixed-length codewords, easy to decode:

```
00001    00010    00000    00011
```

Decoding

- With a fixed-length codewords, easy to decode:

```
00001    00010    00000    00011
```

- How to decode varying-length codes?

Prefix(-Free) Code

Make sure no code word is a prefix of another:

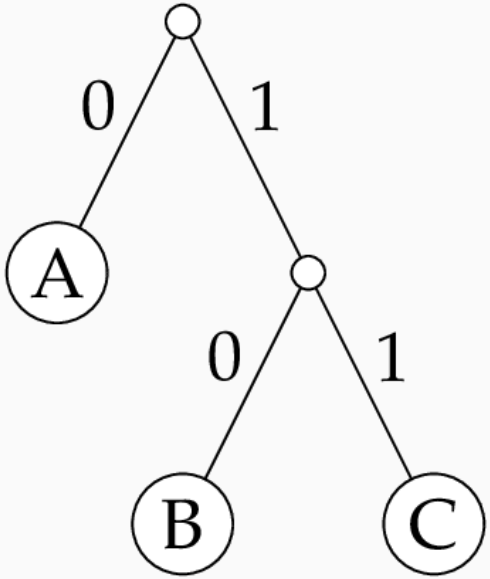
`A = 0, B = 10, C = 11`

`11010001011` **uniquely** decodes into CABAABC

Prefix Codes as Binary Trees

Every prefix code can be converted into a “full” binary tree.

`A = 0, B = 10, C = 11`

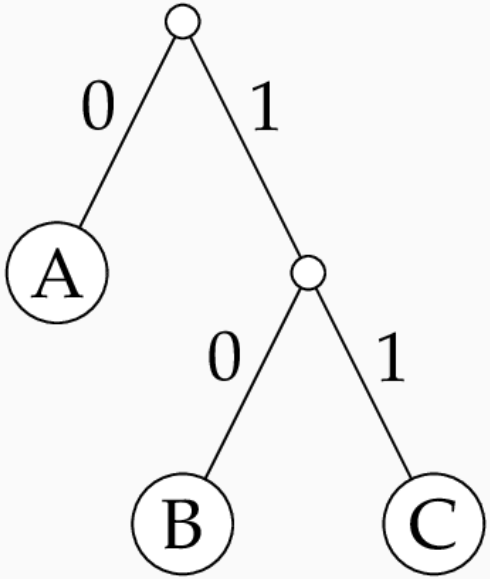


Prefix Codes as Binary Trees

Every prefix code can be converted into a “full” binary tree.

Which chars should be close to the root, and which should be far?

`A = 0, B = 10, C = 11`



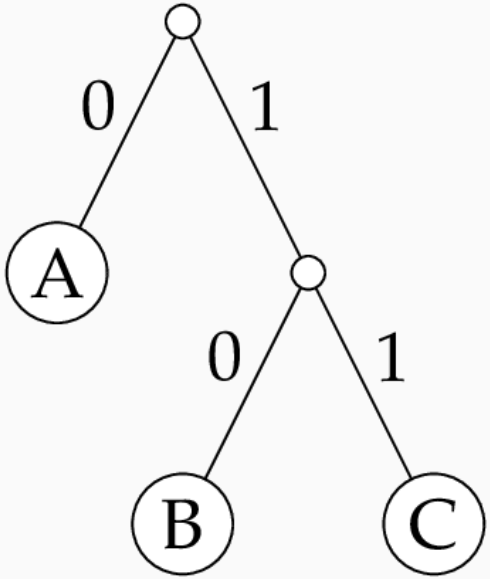
Prefix Codes as Binary Trees

Every prefix code can be converted into a “full” binary tree.

Which chars should be close to the root, and which should be far?

Common = close, uncommon = far

`A = 0, B = 10, C = 11`



Prefix Codes as Binary Trees

“ABCACCBAA”: $f(A) = 4$, $f(B) = 2$, $f(C) = 3$

`A = 0, B = 10, C = 11`

“`ABCACCBAA`” → `01011011111000` (14 bits)

Prefix Codes as Binary Trees

"ABCACCBAA": $f(A) = 4$, $f(B) = 2$, $f(C) = 3$

`A = 0, B = 10, C = 11`

"ABCACCBAA" → `01011011111000` (14 bits)

Claim: best possible encoding length for this word

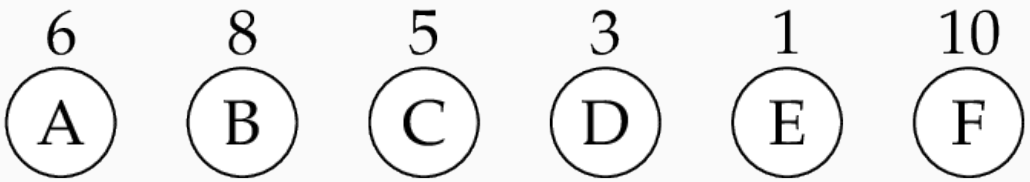
A greedy algorithm

1. Make a node with each character and its frequency.

A greedy algorithm

1. Make a node with each character and its frequency.

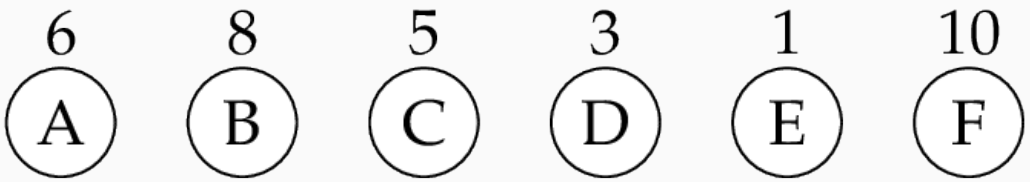
"FFCBCFCBDDAFEBFFBDFBBFBACFFACAABA" →



A greedy algorithm

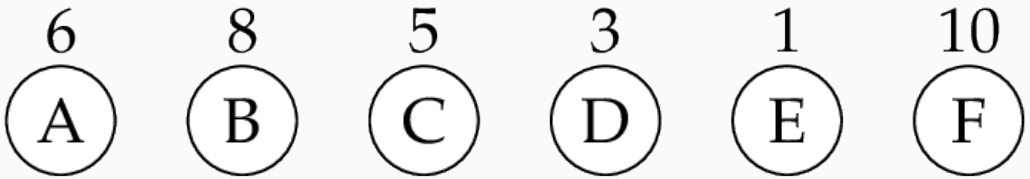
1. Make a node with each character and its frequency.

   “FFCBCFCBDDAFEBFFBDFBBFBACFFACAABA” →

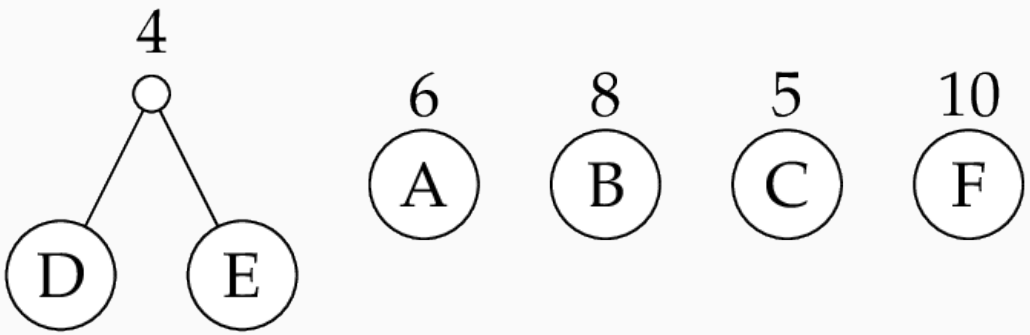


2. Until there’s one tree left:
   a. Make a new tree with the two smallest trees as subtrees, ***adding*** their frequencies.

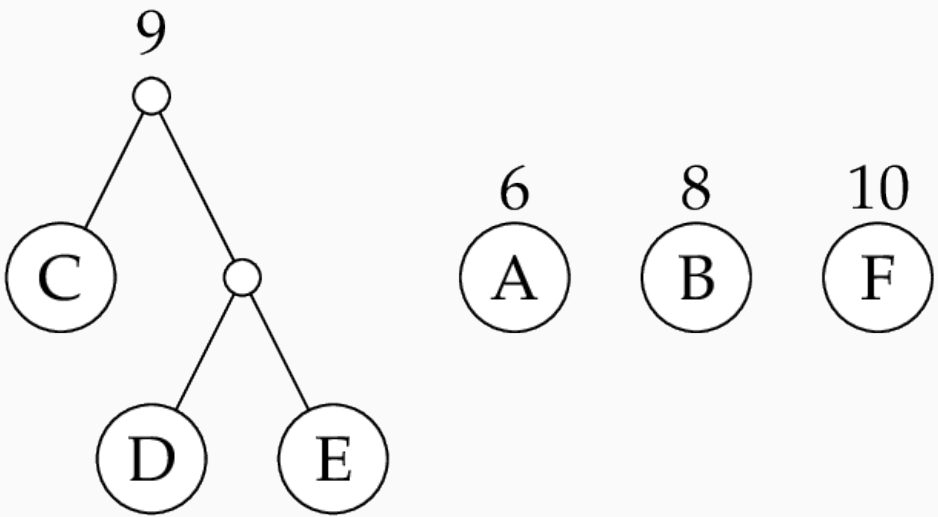
Huffman Coding



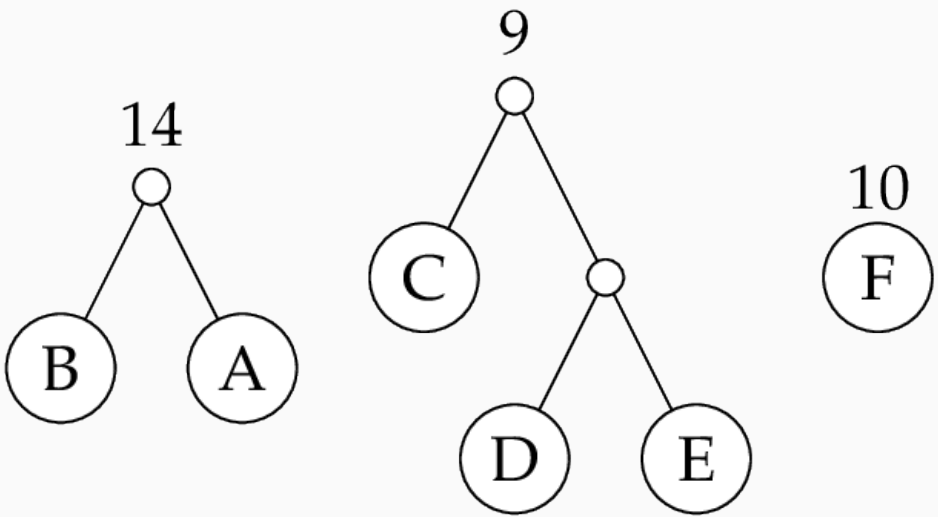
Huffman Coding



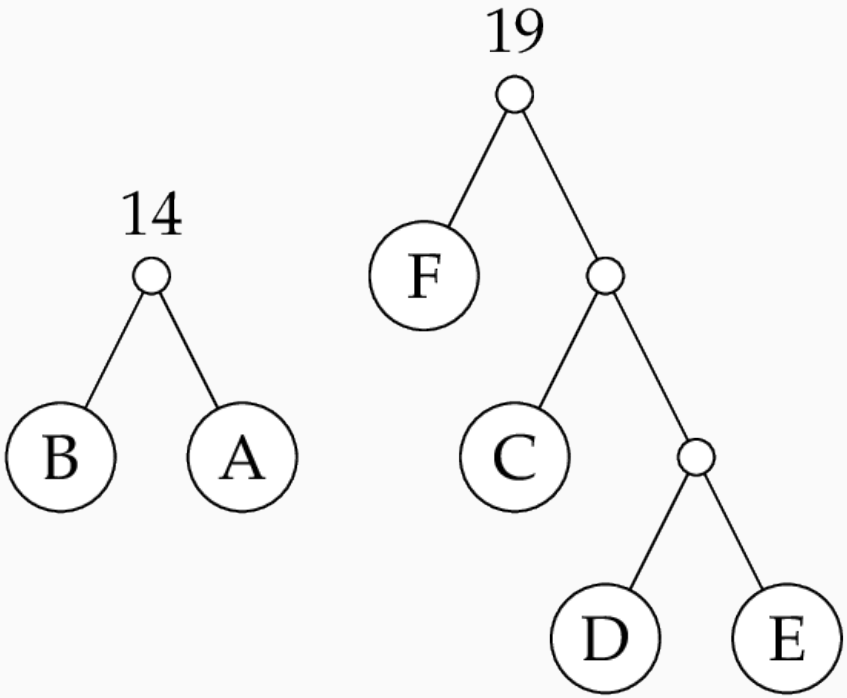
Huffman Coding



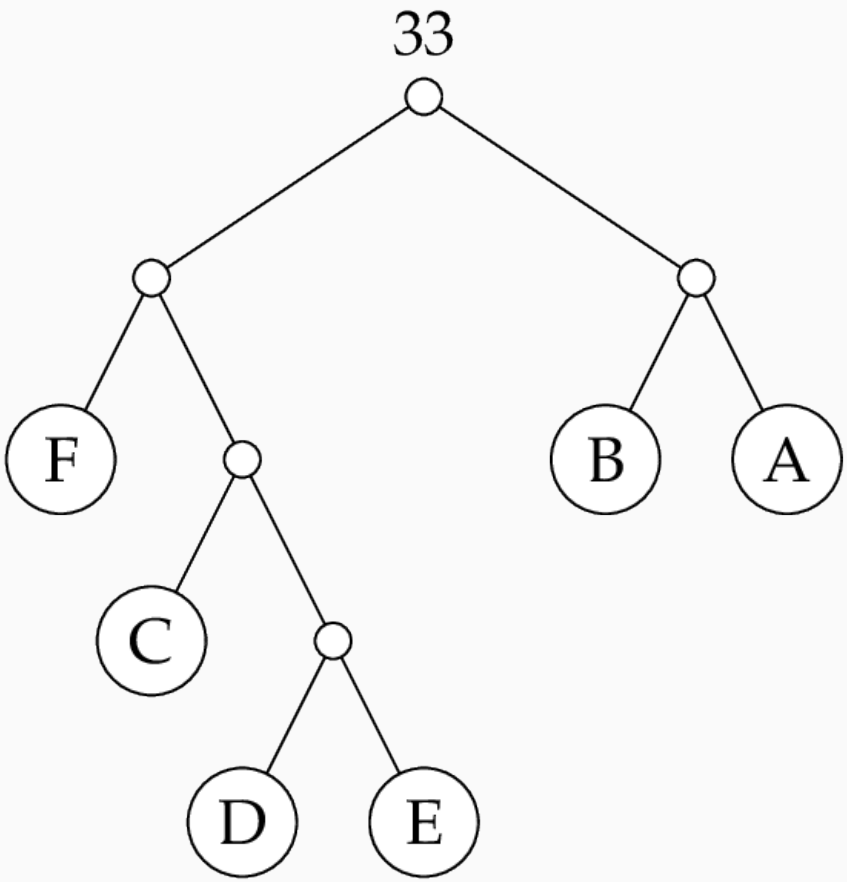
Huffman Coding



Huffman Coding

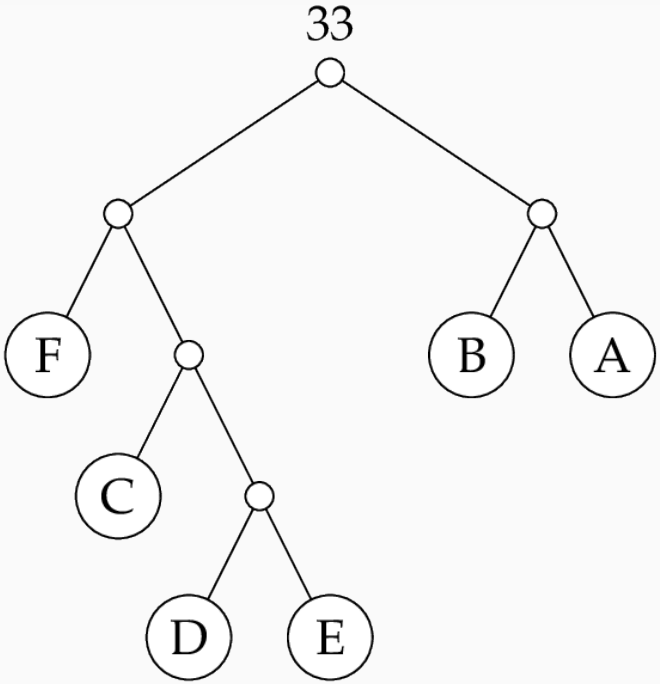


Huffman Coding



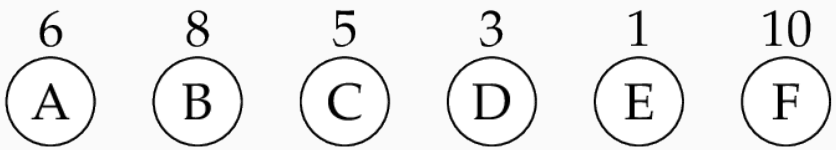
Huffman Coding

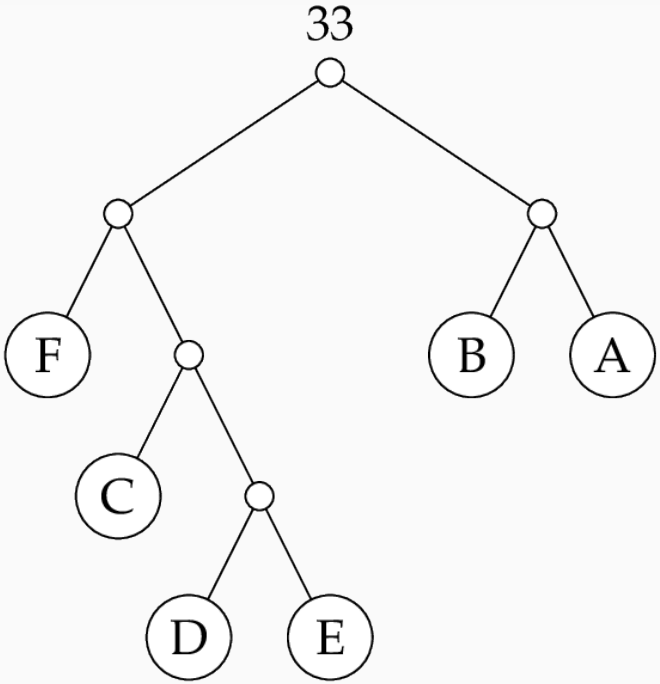
A = 11
B = 10
C = 010
D = 0110
E = 0111
F = 00



Huffman Coding

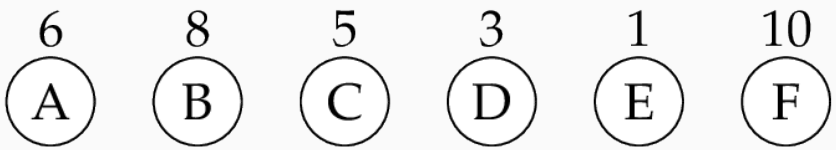
A = 11
B = 10
C = 010
D = 0110
E = 0111
F = 00

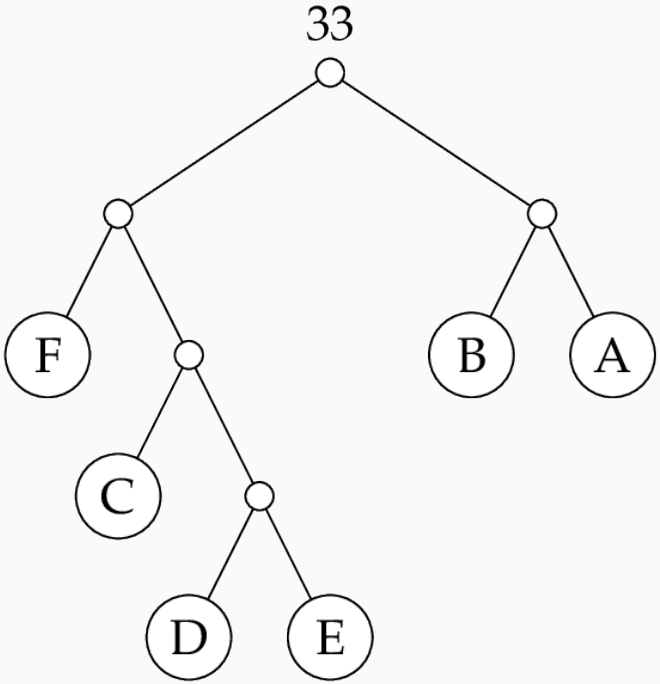




Huffman Coding

A = 11
B = 10
C = 010
D = 0110
E = 0111
F = 00



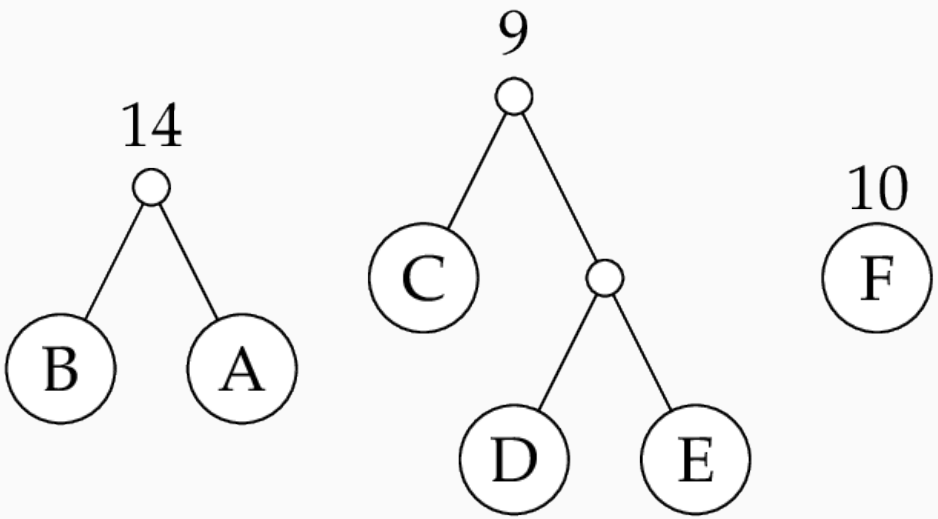


Total length:

6(2)+8(2)+5(3)+3(4)+1(4)+10(2) = 78

Huffman Coding

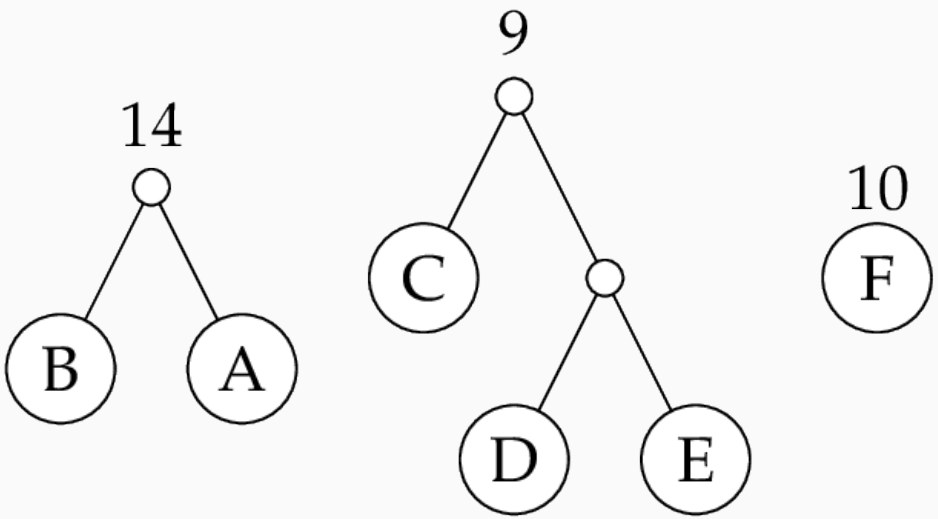
How do we efficiently determine the least frequent trees?



Huffman Coding

How do we efficiently determine the least frequent trees?

Use a priority queue



Huffman Coding

In a binary heap, insert and extract-min take O(log ***n***) time.

```
HUFFMAN(f[1, ..., n]):
    H := HEAP()
    for i = 1, ..., n
        H.INSERT(NODE(i), f[i])
    while |H| > 1
        (t_1, f_1) := H.EXTRACTMIN()
        (t_2, f_2) := H.EXTRACTMIN()
        root := NODE()
        root.LEFT := t_1
        root.RIGHT := t_2
        H.INSERT(root, f_1 + f_2)
    return H.EXTRACTMIN()
```

Huffman Coding

In a binary heap, insert and extract-min take O(log ***n***) time.

Total runtime: O(***n*** log ***n***) time

```
HUFFMAN(f[1, ..., n]):
    H := HEAP()
    for i = 1, ..., n
        H.INSERT(NODE(i), f[i])
    while |H| > 1
        (t_1, f_1) := H.EXTRACTMIN()
        (t_2, f_2) := H.EXTRACTMIN()
        root := NODE()
        root.LEFT := t_1
        root.RIGHT := t_2
        H.INSERT(root, f_1 + f_2)
    return H.EXTRACTMIN()
```

Huffman Coding

Huffman codes try to optimize total codeword length:

$$\sum_{c \in C} f(c) d(c)$$

Huffman Coding

Huffman codes try to optimize total codeword length:

$$\sum_{c \in C} f(c)d(c)$$

Each character **c** is converted into a codeword of length *d*(**c**), which appears *f*(**c**) times.

Huffman Coding

Huffman codes try to optimize total codeword length:

$$\sum_{c \in C} f(c)d(c)$$

Each character ***c*** is converted into a codeword of length *d*(**c**), which appears *f*(**c**) times.

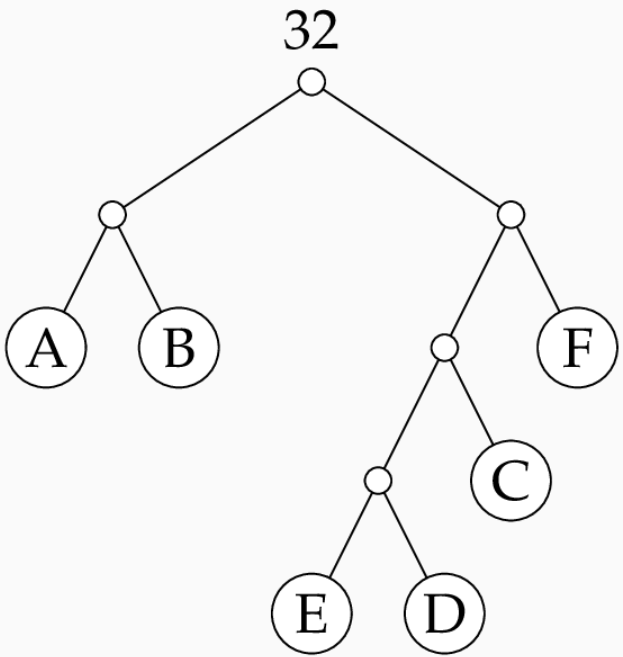
Huffman's algorithm produces an ***optimal*** prefix code.

(but not every optimal code comes from Huffman's algorithm)

“Optimality”

No assumptions about input besides **frequencies**:

- QU in English



“Optimality”

Better compression for the following string:

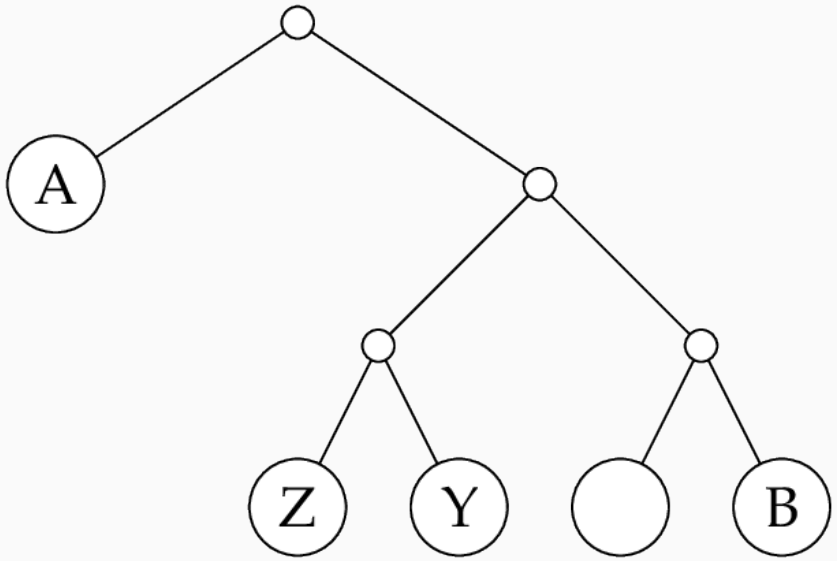
“AAAAAABBCCCCDAAAAAABBCCCCDAAAAAABBCCCCDAAAAAABBCCCCD AAAAAABBCCCCDAAAAAABBCCCCDAAAAAABBCCCCDAAAAAABBCCCCDA AAAAABBCCCCD”

“AAAAAABBCCCCD 9 times”

Correctness

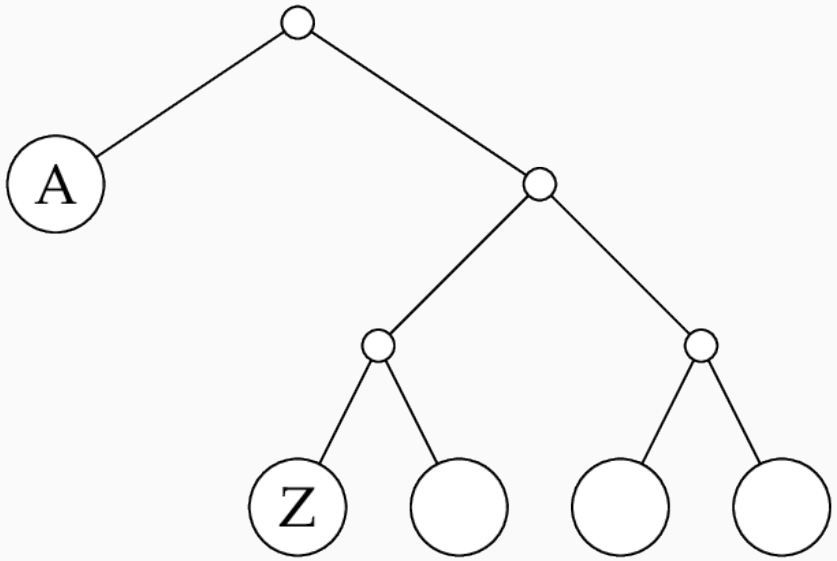
Suppose A and B are the two least common characters.

Claim: In some optimal tree, A and B are siblings (greedy choice).



Greedy choice

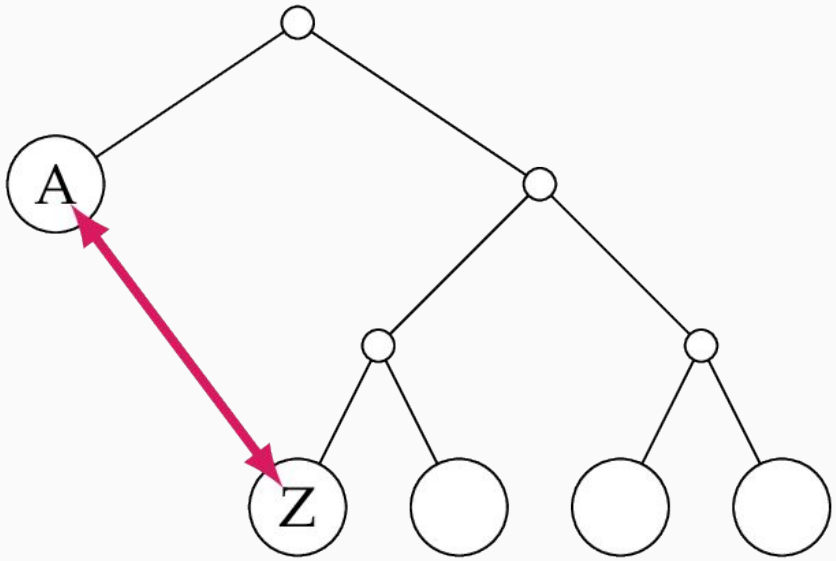
If A is not at the deepest layer, how to improve the cost?



Greedy choice

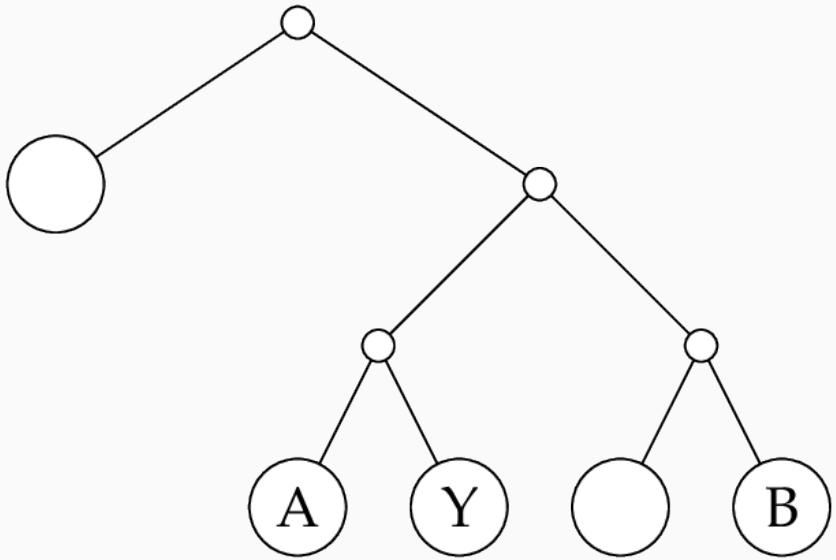
If A is not at the deepest layer, how to improve the cost?

Swap A to the deepest layer.



Greedy choice

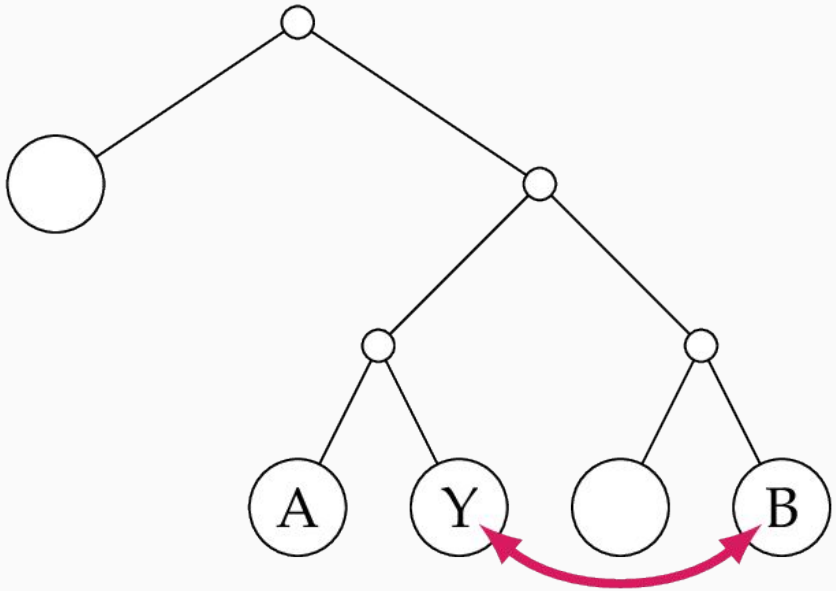
What if A and B are not siblings?



Greedy choice

What if A and B are not siblings?

Do another swap (doesn't change cost).

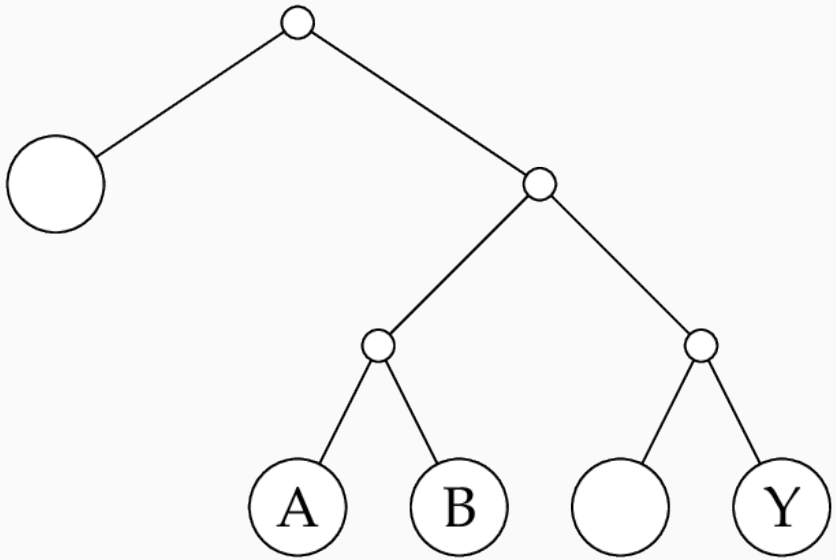


Greedy choice

What if A and B are not siblings?

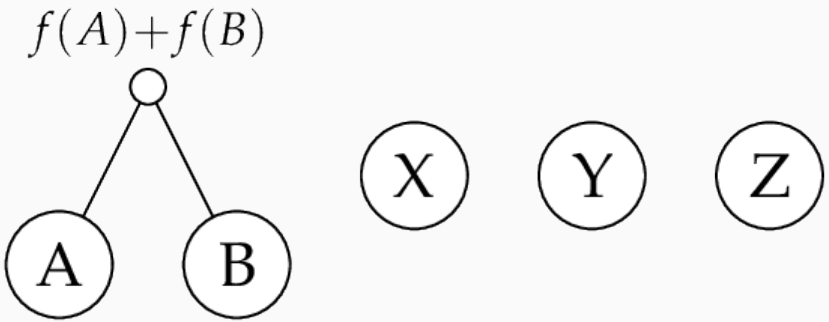
Do another swap (doesn't change cost).

Now they're siblings.



Inductive Step?

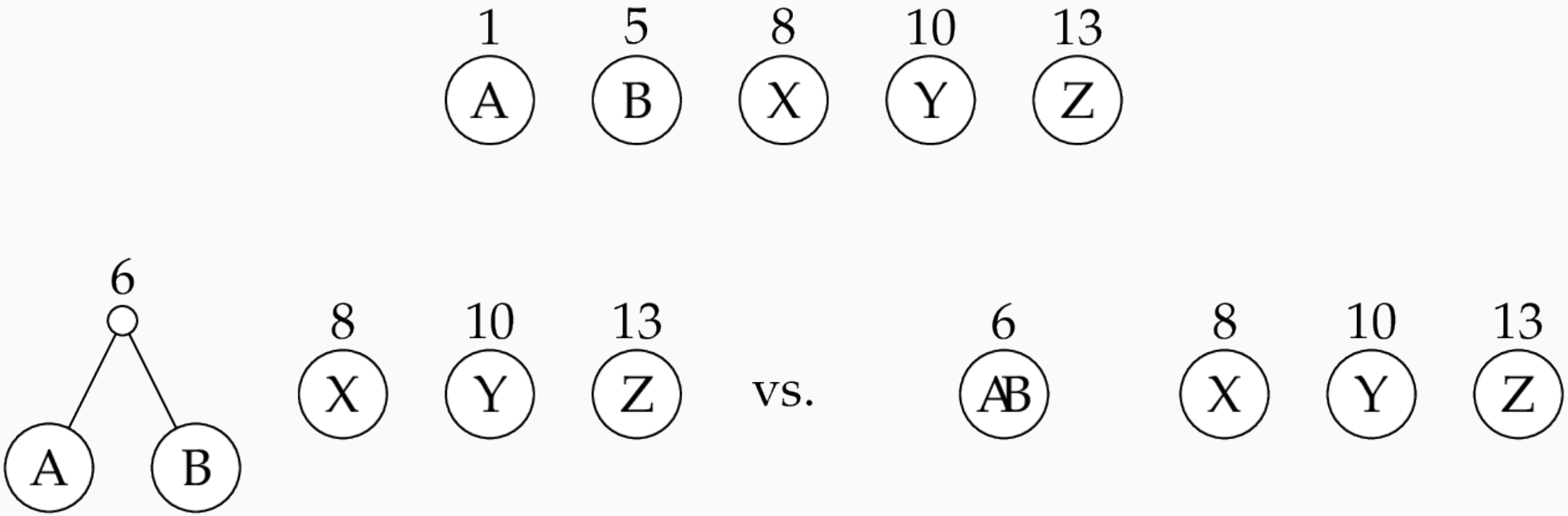
These two cases are not exactly the same...



vs.

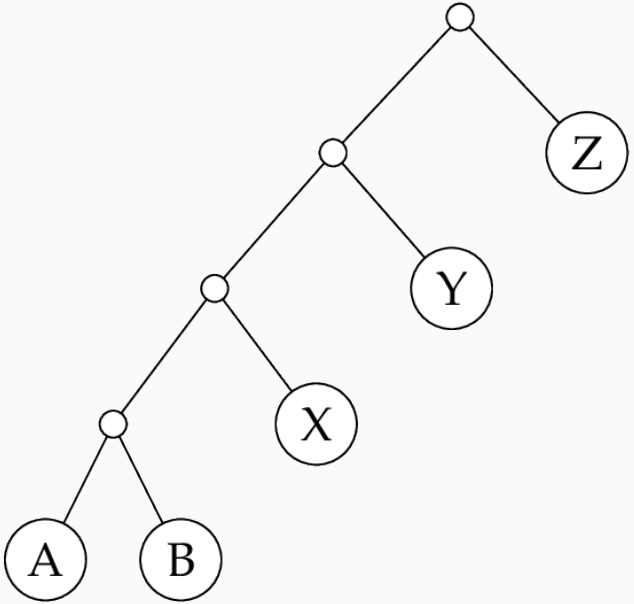
Inductive Step?

These two cases are not exactly the same...

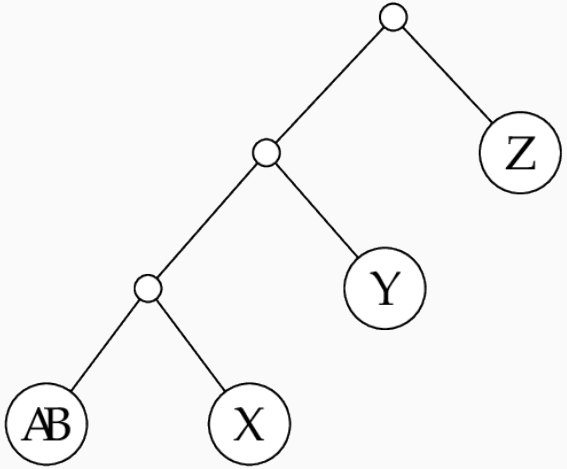


Inductive Step?

Cost: 81



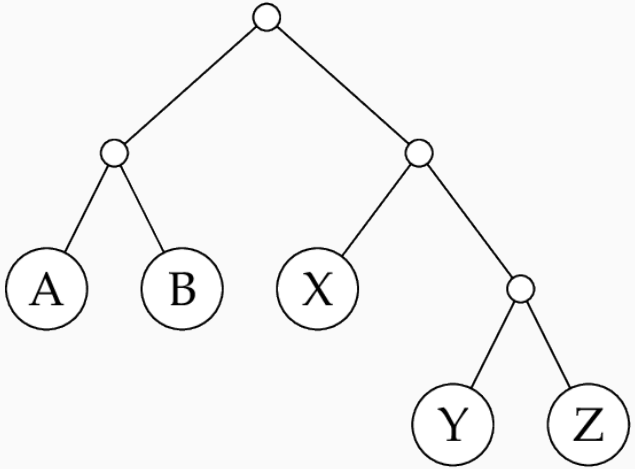
vs.



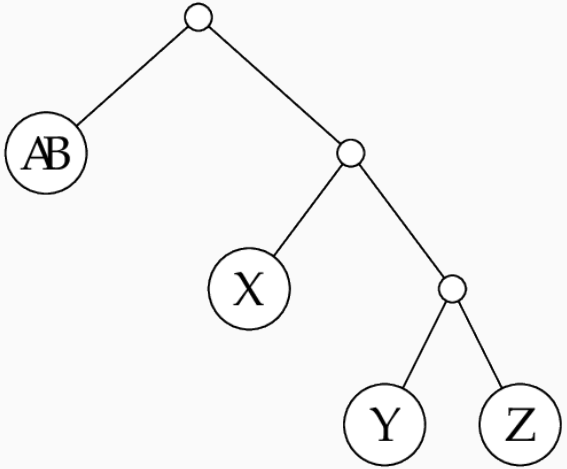
Cost: 75

Inductive Step?

Cost: 97



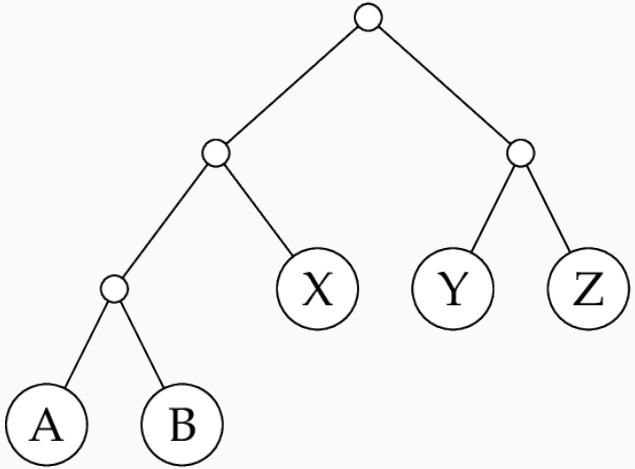
vs.



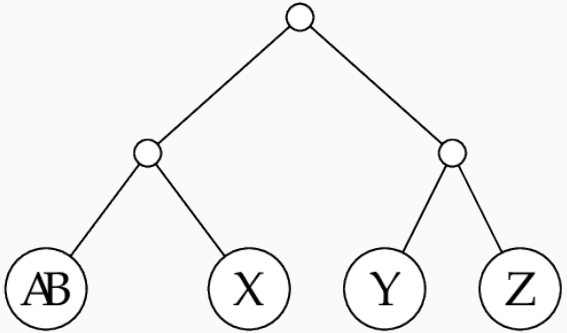
Cost: 91

Inductive Step?

Cost: 80



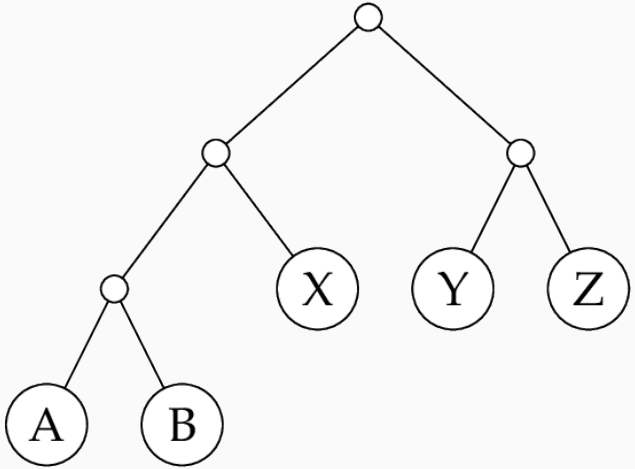
vs.



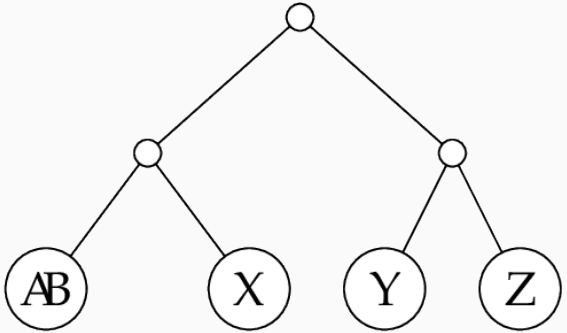
Cost: 74

Inductive Step?

Cost: 80



vs.



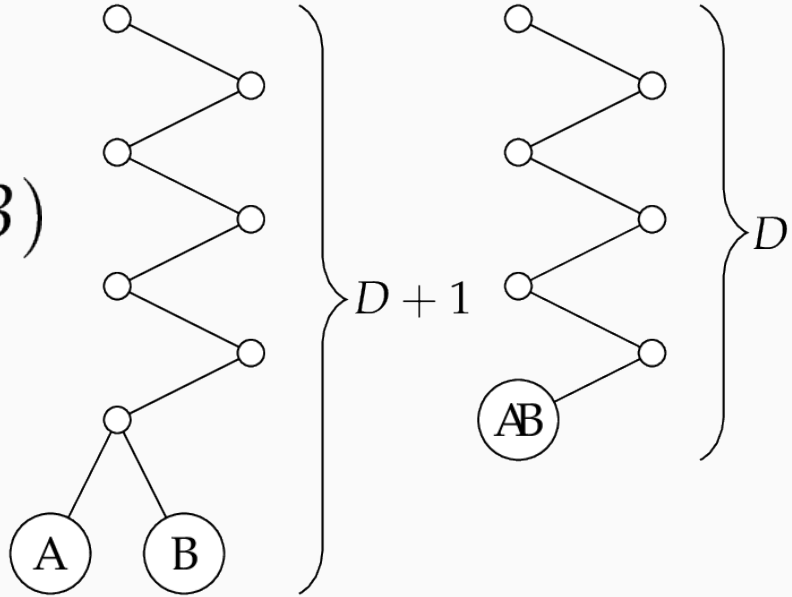
Cost: 74

Tree and cousin always differ by same amount

One final calculation

The difference in cost in any pair of trees is:

$$cost(A) + cost(B) - cost(AB)$$
$$= f(A) + f(B)$$

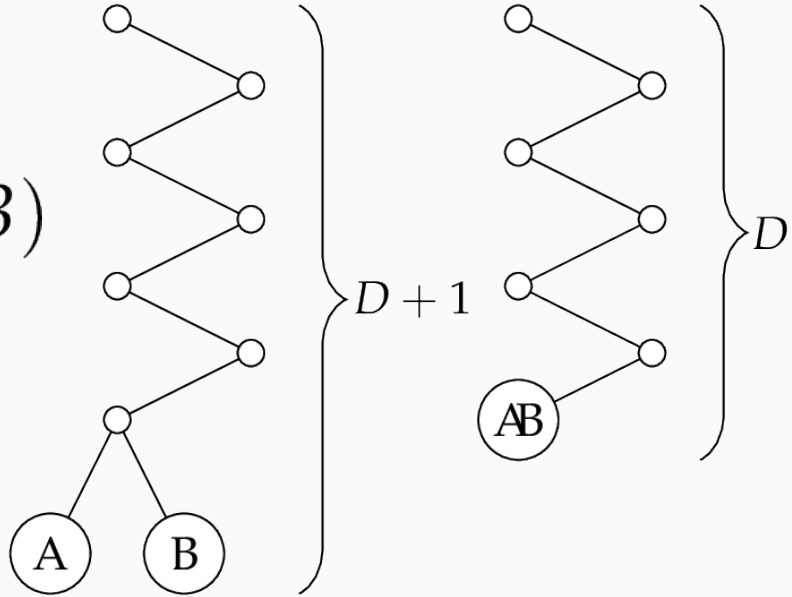


One final calculation

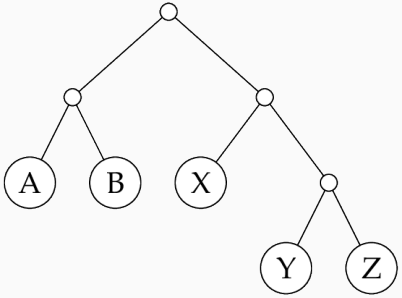
The difference in cost in any pair of trees is:

$$cost(A) + cost(B) - cost(AB)$$
$$= f(A) + f(B)$$

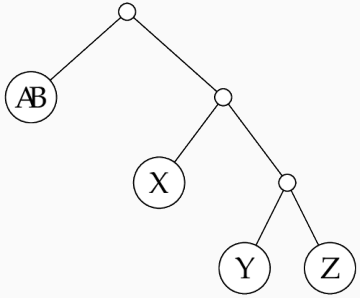
(a value independent of the tree)



Cost: 97

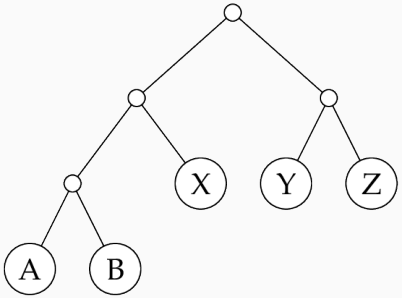


vs.

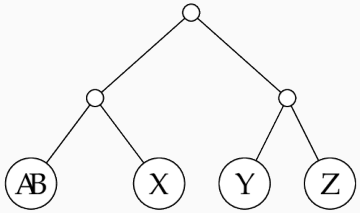


Cost: 91

Cost: 80

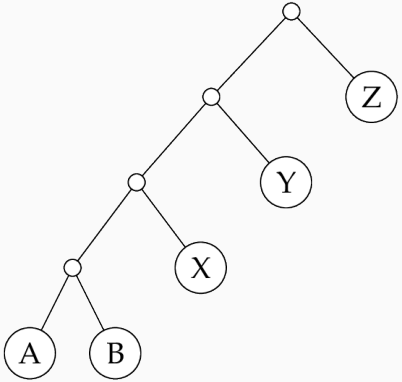


vs.

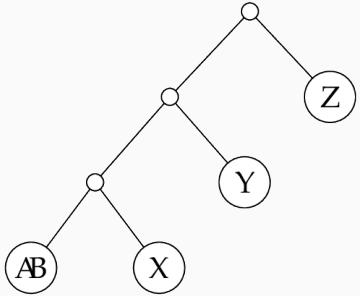


Cost: 74

Cost: 81



vs.



Cost: 75